Points of Interest as Crime Generators: A Supervised Regression Analysis of London Ward-Level Crime Density

> *I, Emily Deeb, pledge my honour that the work presented in this assessment is my own. Where information has been derived from other sources, I confirm that this has been indicated in the work.*

> ***Student Number:*** *25127081*

Preparation

- Github link
- **Number of words:** 1,516 words
- **Runtime:** ~1 minute (pre-processed data loaded from file; full data extraction and model training estimated ~45–60 minutes on first run)

Coding environment: Miniconda 3 · Python 3.x · environment.yml
Hardware: Intel i7-10700 @ 2.90GHz · 10GB RAM
Key packages: geopandas, scikit-learn, shap, osmnx, matplotlib, plotly

To reproduce: `conda env create -f environment.yml` *then* `conda activate urbsim`

In [46]:
```
import time
start_time = time.time()
```

In [47]:
```
# Step 01: Imports + BASE URL
import os
import tempfile
import warnings
import zipfile

import geopandas as gpd
import matplotlib.patches as mpatches
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import numpy as np
import pandas as pd
import plotly.graph_objects as go
import requests
import seaborn as sns
import shap

from io import BytesIO
from matplotlib.colors import LinearSegmentedColormap
from matplotlib.gridspec import GridSpec
from matplotlib.lines import Line2D
from PIL import Image as PILImage
from sklearn.ensemble import RandomForestRegressor
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, root_mean_squared_error
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
from statsmodels.nonparametric.smoothers_lowess import lowess
from IPython.display import Image, display

import osmnx as ox

warnings.filterwarnings("ignore")

BASE = "https://raw.githubusercontent.com/EmilyDeeb/0006_Assessment/main/data/"
```

Table of contents

1. Introduction
2. Research questions
3. Data

4. Methodology
5. Results and discussion
6. Limitations
7. Conclusion
8. References

List of Figures

- Figure 1: Effect of log(x+1) transformation on crime density distributions
- Figure 2: Skewness before and after log(x+1) transformation — POI density features
- Figure 3: Spatial distribution of crime and amenity density across London wards
- Figure 4: Pearson correlation between POI density and crime density by category
- Figure 5: POI feature intercorrelation matrix
- Figure 6: Linearity check — top POI features vs Type A crime density
- Figure 7: MLR standardised coefficients — Type A person and society crime
- Figure 8: MLR standardised coefficients — Type B property crime
- Figure 9: KNN cross-validated R^2 by k
- Figure 10: SHAP feature contributions — Type A person and society crime
- Figure 11: SHAP feature contributions — Type B property crime
- Figure 12: Actual vs predicted — Type A person and society crime density
- Figure 13: Spatial residuals — Type A person and society crime

List of Tables

- Table 1: Crime category definitions and offence types
- Table 2: Spatial and demographic data sources
- Table 3: Crime density descriptive statistics — pre-feature engineering
- Table 4: Points of interest extracted from OpenStreetMap
- Table 5: POI density statistics by category
- Table 6: Variable definitions, sources, units, and roles
- Table 7: Model sequence, role, and justification for method selection.
- Table 8: Model performance on held-out test set
- Table 9: RF 5-fold cross-validated R^2 vs held-out test R^2
- Table 10: Permutation importance for Type A crime

Introduction

[go back to the top]

This study asks whether the spatial layout of urban amenities in London — cafés, clubs, restaurants, etc. — predicts crime density, and whether certain types of amenities are tied to specific categories of crime. London wards is our unit of analysis: their boundaries directly shape police resource allocation and community safety planning. As a result, ward-level predictions offer both operational relevance and academic value, highlighting potential vulnerabilities and identifying zones where intervention may be most influential.

Routine Activity Theory (Cohen and Felson, 1979) argues that crime occurs when motivated offenders, suitable targets, and the absence of capable guardians intersect in the same place and time. This convergence can be defined by the functional land-use profile of an area; the mix of amenities, services, and commercial activity that determines who is present and when. Brantingham and Brantingham (1995) formalise this with the concept of crime generators, theorising that locations that attract large numbers of people for non-criminal reasons increase crime opportunities by providing many possible offenders and targets.

POI data has become a key resource in urban crime research, with Cichosz (2020) demonstrating that OSM POI density features outperform traditional land-use classifications, and Malleson and Andresen (2016) arguing that ambient population — proxied here by POI density- is a more meaningful crime denominator than residential population.

Although significant progress has been made in analysing the relationship between crime and urban fabric, most studies model total crime or individual crime types in isolation, mixing together opportunity structures that operate differently by crime type, a distinction this study wishes to test. To address this gap, we split the crime dataset into two theoretically motivated categories: Type A (person/society crime) and Type B (property crime), and apply SHAP decomposition to compare which amenity types drive each, enabling a more granular account of how urban land use shapes different forms of crime across London wards.

Research question

[go back to the top]

Primary: To what extent does the density of urban amenities predict person/society and property crime densities across London wards?

Sub-question: Which specific amenity categories serve as the primary crime generators for each crime type, and do these differ systematically between person/society and property crime?

Data and Feature Engineering

[go back to the top]

Three datasets were assembled to address these questions.

Dataset 01: Metropolitan Police Service recorded crime data

Obtained from the London Datastore (data.london.gov.uk), containing monthly crime counts by ward and crime category for the 24 months to February 2025. Crimes were summed across the full period and aggregated into two theoretically motivated categories, as seen in Table 1:

Table 1: Crime category definitions and offence types included in the analysis

Crime Category	Type	Offence Types Included	Total Crimes Recorded
Type A — Person/Society Crime	Target variable	Violence against the person, robbery, sexual offences, possession of weapons, drug offences, public order offences, miscellaneous crimes against society, fraud and forgery	813,720 crimes
Type B — Property Crime	Target variable	Theft, burglary, vehicle offences, arson and criminal damage	968,439 crimes

Note: Type B (property crime) exceeds Type A (person/society crime) by approximately 19%.

In [48]:
```
# Data 01: Crime data
# Load crime → aggregate Type A + Type B → crime densities
# Sum monthly crime counts across all 12 months to get annual crime counts,
# then divide by area to get crime densities.
```

In [49]:
```
# Load crime → aggregate Type A + Type B → crime densities
crime_raw = pd.read_csv(BASE + "crime.csv")

# Type A — Person/Society crime categories
typeA_cats = [
    'VIOLENCE AGAINST THE PERSON', 'ROBBERY',
    'SEXUAL OFFENCES', 'POSSESSION OF WEAPONS',
    'DRUG OFFENCES', 'PUBLIC ORDER OFFENCES',
    'MISCELLANEOUS CRIMES AGAINST SOCIETY', 'FRAUD AND FORGERY'
]

# Type B — Property crime
typeB_cats = [
    'THEFT', 'BURGLARY',
    'VEHICLE OFFENCES', 'ARSON AND CRIMINAL DAMAGE'
]
```

In [50]:
```
# Sum monthly columns to annual count
month_cols = [c for c in crime_raw.columns if str(c).startswith("2")]
crime_raw["annual_count"] = crime_raw[month_cols].sum(axis=1)

def aggregate_crime(df, categories, label):
    """Filter to crime categories and sum to ward level."""
    return (
```

```
        df[df["MajorText"].isin(categories)]
        .groupby(["WardCode", "WardName", "LookUp_BoroughName"])["annual_count"]
        .sum()
        .reset_index()
        .rename(columns={
            "annual_count": label,
            "LookUp_BoroughName": "Borough"
        })
    )

crime_A = aggregate_crime(crime_raw, typeA_cats, "typeA_count")
crime_B = aggregate_crime(crime_raw, typeB_cats, "typeB_count")

ward_crime = crime_A.merge(
    crime_B[["WardCode", "typeB_count"]], on="WardCode", how="left"
).fillna(0)

# print(f"Wards: {len(ward_crime)}")
# ward_crime.head()
```

Dataset 02: Ward Spatial Boundary and population We use this dataset to normalise crime counts by ward area and filter the analysis to Greater London. Ward boundary polygons and population estimates were obtained from the ONS Open Geography Portal, as shown in Table 2.

Table 2: Spatial and demographic data sources

Dataset	Source	URL	Geography	Notes
Ward boundary polygons	ONS Open Geography Portal	geoportal.statistics.gov.uk	Ward level	2024 boundaries, filtered to London using GLA boundary
Ward areas	Derived from boundary polygons	—	Ward level	Calculated using British National Grid projection (EPSG:27700)
Population estimates	ONS mid-2021 ward-level estimates	geoportal.statistics.gov.uk	Ward level	No missing values after merging; 679 London wards retained

Note: Population density was retained in the dataset but excluded from model features to isolate the contribution of amenity configuration independently of population size. Incorporating it as a control variable is a natural next step.

In [51]:
```
# Data 02: Ward Spatial Data
# load population and area data
# load GLA boundary + filter 2024 wards to London
# Centroid of each ward
# spatial join to GLA boundary to filter to London wards
# calculate area in km2
# Build main GeoDataFrame -> compute crime densities using ward geometries + crime counts
# ==============================================================================
# Table 3: Crime density descriptive stats
```

In [52]:
```
# Load population
pop = pd.read_csv(BASE + "ward_population.csv", encoding="latin-1")

# Rename columns and select relevant ones
pop = pop.rename(columns={
    "Ward 2023 Code": "WardCode",
    "Total": "population"
})[["WardCode", "population"]]

# Remove commas and convert population to int
pop["population"] = (
    pop["population"].astype(str)
    .str.replace(",", "")
    .astype(int)
)

# Filter to E05 wards only (Greater London)
pop = pop[pop["WardCode"].str.startswith("E05")].copy()

# Merge population data with crime data
ward_crime = ward_crime.merge(pop, on="WardCode", how="left")

# Check for missing population values after merge
# print(f"Missing population: {ward_crime['population'].isna().sum()}")
```

In [53]:
```
# ── Load 2024 ward boundaries (GeoJSON) ──────────────────────
wards_2024 = gpd.read_file(BASE + "wards_2024.geojson")

# ── Load GLA boundary (shapefile — requires all component files) ─
gla_files = [
    "London_GLA_Boundary.dbf",
```

```
    "London_GLA_Boundary.prj",
    "London_GLA_Boundary.shp",
    "London_GLA_Boundary.shx",
]

gla_dir = tempfile.mkdtemp()
for fname in gla_files:
    url = BASE + "gla/" + fname
    r = requests.get(url)
    with open(os.path.join(gla_dir, fname), "wb") as f:
        f.write(r.content)

gla_gdf = gpd.read_file(os.path.join(gla_dir, "London_GLA_Boundary.shp"))
gla_gdf = gla_gdf.to_crs("EPSG:4326")

# ── Compute ward centroids ──────────────────────────────────────
wards_centroids = wards_2024.copy()
wards_centroids["geometry"] = (
    wards_2024.to_crs("EPSG:27700").geometry.centroid.to_crs("EPSG:4326")
)
wards_centroids = gpd.GeoDataFrame(
    wards_centroids, geometry="geometry", crs="EPSG:4326"
)

# print(f"GLA boundary loaded: {len(gla_gdf)} features")
# print(f"Wards 2024 loaded: {len(wards_2024)} wards")
```

In [54]:

```
# Spatial join to filter to wards whose centroids fall within GLA boundary
london_wards = gpd.sjoin(
    wards_centroids,
    gla_gdf[["geometry"]],
    how="inner", predicate="within"
)

# Calculate area in km2 using projected CRS for accurate area calculation
london_wards = wards_2024.loc[london_wards.index].copy()
london_wards = london_wards.rename(columns={
    "WD24CD": "WardCode",
    "WD24NM": "WardName_geo"
})

# Remove non-geographic wards
london_wards = london_wards[
    ~london_wards["WardName_geo"].str.contains(
        'Aviation|SO18', case=False, na=False
    )
].copy()

# Calculate area in km2 using projected CRS for accurate area calculation
london_wards["area_km2"] = (
    london_wards.to_crs("EPSG:27700").geometry.area / 1_000_000
)

# Check final wards + area
# london_wards[['WardCode', 'WardName_geo', 'area_km2']].head()
```

In [55]:

```
# print(f"Ward count after filter: {len(london_wards)}")
```

In [56]:

```
# Note: 25 wards in the 2024 boundary file have no matching crime records
# (E05009288–E05009312) — likely new ward codes created in the 2024 boundary
# revision that postdate the MPS crime dataset. These are dropped automatically
# in Step 05 via the left merge and dropna on geometry/area_km2.
# Final analysis uses 679 matched wards.
# print(f"Ward count after filter: {len(london_wards)}")
# print("Note: 25 unmatched wards will be dropped at merge stage — expected behaviour")
```

In [57]:

```
# Build main GeoDataFrame
# Merge crime counts with ward geometries
ward_gdf = ward_crime.merge(
    london_wards[["WardCode", "area_km2", "geometry"]],
    on="WardCode",
    how="left",
)

# Compute crime density (target variables)
ward_gdf["typeA_density"] = ward_gdf["typeA_count"] / ward_gdf["area_km2"]
ward_gdf["typeB_density"] = ward_gdf["typeB_count"] / ward_gdf["area_km2"]

# Promote to GeoDataFrame and drop rows with missing spatial data
ward_gdf = (
    gpd.GeoDataFrame(ward_gdf, geometry="geometry", crs="EPSG:4326")
    .dropna(subset=["geometry", "area_km2"])
)

# print(f"Final ward count:    {len(ward_gdf)}")
# print(f"Missing geometries:  {ward_gdf['geometry'].isna().sum()}")

# ── Styled summary: crime density descriptive stats (pre-transformation) ───

STYLE = {
    "font-family": "Inter, sans-serif",
    "font-size":   "12px",
    "color":       "#1B2632",
```

```
    "border-bottom": "1px solid #C9C1B1",
    "padding":     "7px 12px",
}

TABLE_STYLES = [

    {
        "selector": "",
        "props": [
            ("margin-left", "auto"),
            ("margin-right", "auto"),
        ],
    },

    {
        "selector": "thead tr",
        "props": [
            ("background-color", "#2C3B4D"),
            ("color",            "#F7F4ED"),
            ("font-weight",      "500"),
        ],
    },
    {
        "selector": "caption",
        "props": [
            ("font-size",   "11px"),
            ("color",       "#F7F4ED"),
            ("font-style",  "italic"),
            ("text-align",  "left"),
            ("padding",     "6px 0"),
        ],
    },
]

(
    ward_gdf[["typeA_density", "typeB_density"]]
    .describe()
    .round(2)
    .style
    .format("{:.2f}")
    .set_properties(**STYLE)
    .set_table_styles(TABLE_STYLES)
)
```

Out[57]:

	typeA_density	typeB_density
count	679.00	679.00
mean	868.35	1053.34
std	765.14	1505.26
min	8.47	8.56
25%	322.69	304.79
50%	619.01	596.75
75%	1199.32	1237.23
max	4978.44	24964.44

Table 3: Crime density descriptive statistics (crimes per km²) — pre-feature engineering. *Raw crime density is unsuitable for direct modelling: both variables are strongly right-skewed, with standard deviations above their means (* `typeA_density` *: σ=765, μ=868;* `typeB_density` *: σ=1505, μ=1053). Crime concentrates in a few high-density wards, inflating upper values, especially for property crime. A log transformation is applied next to address this.*

Dataset 03: Point-of-interest data were extracted from OpenStreetMap using the OSMnx Python library (Boeing, 2017). After filtering to 17 predefined categories selected as proxies for footfall and economic activity, 41,502 POIs were retained. Table 4 below shows each category, its OSM tag, and total count.

In [58]:

```
# Data 03: Point of Interest (POI) Data
# Download POIs from OpenStreetMap (points and polygons centroids)
# Clean Point data → filter noise → spatial join to GLA boundary → compute densities
# ==============================================================================
# Table 4: POI categories, OSM tags, and counts
# Table 5: POI density descriptive stats
```

In [59]:

```
# Download POIs from OpenStreetMap
gla_polygon = gla_gdf.to_crs("EPSG:4326").union_all()

tags = {
    "amenity": ["pub", "bar", "nightclub", "fast_food", "restaurant",
                "cafe", "bank", "atm", "pharmacy"],
    "railway": ["station"],
    "shop": ["supermarket", "convenience", "mall", "department_store",
             "clothes", "electronics", "jewelry"]
}
```

```
print("Downloading POIs — this takes 5–10 minutes...")
poi_raw = ox.features.features_from_polygon(gla_polygon, tags=tags) #, which returns a GeoDataFrame of all n

# Include both points AND polygon centroids
# Many venues (pubs, restaurants) are mapped as building polygons in OSM
poi_points = poi_raw[poi_raw.geometry.type == "Point"].copy()
poi_polys  = poi_raw[poi_raw.geometry.geom_type.isin(["Polygon", "MultiPolygon"])].copy()
poi_polys["geometry"] = poi_polys.to_crs("EPSG:27700").geometry.centroid.to_crs("EPSG:4326")

poi_gdf = pd.concat([poi_points, poi_polys], ignore_index=True)

# Extract poi_type — check amenity, shop, railway in priority order
poi_gdf["poi_type"] = (
    poi_gdf["amenity"]
    .fillna(poi_gdf.get("shop", pd.NA))
    .fillna(poi_gdf.get("railway", pd.NA))
)

poi_gdf = poi_gdf[["geometry", "poi_type"]].dropna()
poi_gdf = poi_gdf.set_crs("EPSG:4326")

# print(f"Total POIs (points + polygon centroids): {len(poi_gdf)}")

# print(poi_gdf["poi_type"].value_counts())
# We have extra points because some venues are mapped as both points and polygons in OSM. We keep both to co
# remove doctors
```

```
Downloading POIs — this takes 5–10 minutes...
```

In [60]:

```
# Filter noise → spatial join → densities
valid_types = [
    "pub", "bar", "nightclub", "fast_food", "restaurant", "cafe",
    "bank", "atm", "pharmacy", "station",
    "supermarket", "convenience", "mall", "department_store",
    "clothes", "electronics", "jewelry"
]

# Filter to valid POI types only — this removes ~1,000 noisy points (mostly doctors, which are often mapped
poi_gdf = poi_gdf[poi_gdf["poi_type"].isin(valid_types)].copy()
print(f"POIs after cleaning: {len(poi_gdf)}")

# Spatial join to count POIs within each ward
wards_4326 = ward_gdf[["WardCode", "geometry"]].to_crs("EPSG:4326")
poi_in_wards = gpd.sjoin(poi_gdf, wards_4326, how="left", predicate="within")

poi_counts = (
    poi_in_wards
    .groupby(["WardCode", "poi_type"])
    .size()
    .unstack(fill_value=0)
    .reset_index()
)
poi_counts.columns.name = None

# Drop existing POI/density columns if re-running
existing = [c for c in poi_counts.columns if c != "WardCode" and c in ward_gdf.columns]
existing += [c for c in ward_gdf.columns if c.endswith("_density")
             and c not in ["typeA_density", "typeB_density"]]
ward_gdf = ward_gdf.drop(columns=existing, errors="ignore")

# Merge — left join preserves all 679 wards
ward_gdf = ward_gdf.merge(poi_counts, on="WardCode", how="left")

# Ensure columns exist for wards with no POIs
poi_cols = [c for c in poi_counts.columns if c != "WardCode"]
for col in poi_cols:
    if col not in ward_gdf.columns:
        ward_gdf[col] = 0
ward_gdf[poi_cols] = ward_gdf[poi_cols].fillna(0)

# Compute densities
for col in poi_cols:
    ward_gdf[f"{col}_density"] = ward_gdf[col] / ward_gdf["area_km2"]

poi_density_cols = [f"{c}_density" for c in poi_cols]

# Report empty wards
empty = ward_gdf[ward_gdf[poi_cols].sum(axis=1) == 0][["WardCode", "WardName", "Borough"]]
# print(f"Wards with no POIs: {len(empty)}")
# print(empty.to_string(index=False))
# print(f"\nPOI density features: {len(poi_density_cols)}")
```

```
POIs after cleaning: 41502
```

In [61]:

```
# Table : POI categories, OSM tags, and counts
tag_map = {
    'restaurant':       ('Restaurant',        'amenity=restaurant'),
    'cafe':             ('Cafe',              'amenity=cafe'),
    'fast_food':        ('Fast food',         'amenity=fast_food'),
    'convenience':      ('Convenience store', 'shop=convenience'),
    'clothes':          ('Clothes shop',      'shop=clothes'),
    'atm':              ('ATM',               'amenity=atm'),
    'pub':              ('Pub',               'amenity=pub'),
    'pharmacy':         ('Pharmacy',          'amenity=pharmacy'),
    'bar':              ('Bar',               'amenity=bar'),
```

```python
    'supermarket':      ('Supermarket',       'shop=supermarket'),
    'station':          ('Rail station',      'railway=station'),
    'bank':             ('Bank',              'amenity=bank'),
    'jewelry':          ('Jewellery shop',    'shop=jewelry'),
    'electronics':      ('Electronics shop',  'shop=electronics'),
    'nightclub':        ('Nightclub',         'amenity=nightclub'),
    'department_store': ('Department store',  'shop=department_store'),
    'mall':             ('Mall',              'shop=mall'),
}

counts = poi_gdf["poi_type"].value_counts()

rows = []
for poi_type, (label, tag) in tag_map.items():
    count = counts.get(poi_type, 0)
    rows.append({'POI Category': label, 'OSM Tag': tag, 'Count': int(count)})

table3 = pd.DataFrame(rows).sort_values('Count', ascending=False).reset_index(drop=True)

total_row = pd.DataFrame([{
    'POI Category': 'Total',
    'OSM Tag': '',
    'Count': table3['Count'].sum()
}])
table3 = pd.concat([table3, total_row], ignore_index=True)

(table3.style
    .format({'Count': '{:,.0f}'})
    .set_properties(**{
        'font-family': 'Inter, sans-serif',
        'font-size': '13px',
        'text-align': 'left',
        'color': '#1B2632',
        'background-color': '#F7F4ED',
        'border-bottom': '1px solid #C9C1B1',
        'padding': '8px 14px'
    })
    .set_table_styles([
        {
        "selector": "",
        "props": [
            ("margin-left", "auto"),
            ("margin-right", "auto"),
        ],
    },
        {'selector': 'thead tr',
         'props': [('background-color', '#2C3B4D'),
                   ('color', '#F7F4ED'),
                   ('font-weight', '500'),
                   ('font-size', '13px')]},
        {'selector': 'tbody tr:nth-child(even)',
         'props': [('background-color', '#EAE6DE')]},
        {'selector': 'tr:last-child',
         'props': [('font-weight', 'bold'),
                   ('border-top', '2px solid #2C3B4D')]},
        {'selector': 'caption',
         'props': [('font-size', '11px'),
                   ('color', '#F7F4ED'),
                   ('font-style', 'italic'),
                   ('text-align', 'left'),
                   ('padding', '6px 0')]}
    ])
    .hide(axis='index')
)
```

Out[61]:

POI Category	OSM Tag	Count
Restaurant	amenity=restaurant	8,978
Cafe	amenity=cafe	6,718
Fast food	amenity=fast_food	6,158
Convenience store	shop=convenience	5,432
Pub	amenity=pub	3,177
Clothes shop	shop=clothes	2,933
ATM	amenity=atm	1,674
Pharmacy	amenity=pharmacy	1,280
Supermarket	shop=supermarket	1,252
Bar	amenity=bar	1,215
Bank	amenity=bank	703
Jewellery shop	shop=jewelry	695
Rail station	railway=station	616
Electronics shop	shop=electronics	300
Nightclub	amenity=nightclub	154
Department store	shop=department_store	134
Mall	shop=mall	83
Total		**41,502**

Table 4: Points of Interest (POI) Extracted from OpenStreetMap — Greater London. *Food and beverage venues dominate — restaurants, cafés, fast food, pubs, and bars account for 26,246 POIs (63% of total).*

In [62]:

```
# POI density descriptive statistics
# Custom palette for gradient
palette_cmap = LinearSegmentedColormap.from_list(
    'palette', ['#F7F4ED', '#C9C1B1', "#A8696F"]
)

# Build describe table
desc = ward_gdf[poi_density_cols].describe().round(3).T

# Replace count with total POI count per category
desc['count'] = [ward_gdf[col.replace('_density', '')].sum()
                 for col in poi_density_cols]

(desc
    .drop(columns=['min'])
    .rename(columns={'count': 'Total POIs', 'mean': 'Mean', 'std': 'Std',
                     '25%': 'Q1', '50%': 'Median', '75%': 'Q3', 'max': 'Max'})
    .style
    .format({'Total POIs': '{:.0f}', 'Mean': '{:.3f}', 'Std': '{:.3f}',
             'Q1': '{:.3f}', 'Median': '{:.3f}', 'Q3': '{:.3f}', 'Max': '{:.3f}'})
    .background_gradient(subset=['Mean', 'Std', 'Max'], cmap=palette_cmap)
    .set_properties(**{
        'font-family': 'Inter, sans-serif',
        'font-size': '12px',
        'color': '#1B2632',
        'border-bottom': '1px solid #C9C1B1',
        'padding': '7px 12px'
    })
    .set_table_styles([
        {
        "selector": "",
        "props": [
            ("margin-left", "auto"),
            ("margin-right", "auto"),
        ],
    },
        {'selector': 'thead tr',
         'props': [('background-color', '#2C3B4D'),
                   ('color', '#F7F4ED'),
                   ('font-weight', '500')]},
        {'selector': 'caption',
         'props': [('font-size', '11px'),
                   ('color', '#C9C1B1'),
                   ('font-style', 'italic'),
                   ('text-align', 'left'),
                   ('padding', '6px 0')]}
    ])
)
# Note: the high max values for some categories (e.g. cafe, restaurant) indicate that a few wards have very
# likely due to major commercial centers. This skew is important to note when interpreting the data and may
# investigation or transformation in future analyses.
```

Out[62]:

	Total POIs	Mean	Std	Q1	Median	Q3	Max
atm_density	1629	1.866	2.908	0.000	0.739	2.332	20.721
bank_density	643	0.722	1.866	0.000	0.000	0.333	12.158
bar_density	1098	1.372	3.971	0.000	0.000	1.052	47.284
cafe_density	6482	7.582	11.638	0.978	3.226	9.320	99.362
clothes_density	2861	3.547	13.987	0.000	0.000	1.760	191.327
convenience_density	5392	6.256	6.350	1.621	3.904	9.195	35.078
department_store_density	133	0.132	0.483	0.000	0.000	0.000	5.052
electronics_density	300	0.338	0.839	0.000	0.000	0.252	8.769
fast_food_density	5909	6.664	9.457	1.154	3.086	8.116	61.386
jewelry_density	678	0.782	3.937	0.000	0.000	0.074	79.490
mall_density	81	0.102	0.390	0.000	0.000	0.000	3.155
nightclub_density	150	0.195	0.788	0.000	0.000	0.000	9.632
pharmacy_density	1268	1.377	1.662	0.314	0.882	1.900	14.568
pub_density	3047	3.472	5.393	0.575	1.375	4.091	46.971
restaurant_density	8686	10.235	19.567	1.015	3.485	11.093	235.109
station_density	599	0.600	0.858	0.000	0.305	0.914	5.022
supermarket_density	1246	1.333	1.785	0.000	0.708	1.845	13.470

Table 5: POI Density Statistics by Category — Ward-Level Distribution Across Greater London. *Ward-level POI densities (POIs per km²). Restaurant and café densities show the highest means and greatest skew — restaurant_density peaks at 235.1, clothes_density at 191.3, both far exceeding their Q3 values, indicating concentration in a small number of central wards. Sparse categories (mall, department_store, nightclub) have a median of zero, absent from the majority of wards.*

Feature Engineering:

All crime and POI counts were normalised by ward area (per km²) to account for variation in ward size. We then applied a log transformation to all density variables to reduce right skewness before modelling, as shown in Tables 3 and 5.

$$x' = \log\left(\frac{\text{count}_i}{\text{area}_i} + 1\right)$$

> *where x' is the transformed feature, count_i is the raw crime or POI count for ward i, area_i is the ward area in km², and the $+1$ constant prevents undefined values where counts are zero.*

After transformation, residual skewness fell below 1.5 for 12 of the 17 POI features — mall, electronics, department_store, nightclub, and jewellery remaining moderately skewed due to their sparse distribution across wards. Figures 1 and 2 show the effect of this transformation on crime density and POI feature distributions, respectively. Table 6 summarises all variables used in the model.

In [63]:
```
# Feature engineering
# log transform POI densities to reduce skew (add small constant to avoid log(0))
# ==============================================================================
# Figure 1: Effect of log transformation on crime density
# Figure 2: Skewness before and after log transformation
# Figure 3: Spatial distribution of crime and amenity densities
```

In [64]:
```
# Log transform all densities
all_density_cols = ["typeA_density", "typeB_density"] + poi_density_cols

for col in all_density_cols:
    ward_gdf[f"{col}_log"] = np.log1p(ward_gdf[col])

log_feature_cols = [f"{c}_log" for c in poi_density_cols]
log_targetA = "typeA_density_log"
log_targetB = "typeB_density_log"

# Before vs after skewness comparison
skew_before = ward_gdf[poi_density_cols].skew().rename("Before")
skew_after  = ward_gdf[log_feature_cols].skew().rename("After")
skew_after.index = skew_before.index  # align index for comparison

skew_df = pd.concat([skew_before, skew_after], axis=1).sort_values("Before", ascending=False)
```

In [65]:
```
# Distribution plots: raw vs log transformed
fig, axes = plt.subplots(2, 2, figsize=(12, 8), facecolor='#F7F4ED')
```

```
# We will compare the distributions of the raw crime density features (typeA_density, typeB_density) with th
pairs = [
    ("typeA_density", "typeA_density_log",
     "Type A — raw (crimes per km²)", "Type A — log transformed"),
    ("typeB_density", "typeB_density_log",
     "Type B — raw (crimes per km²)", "Type B — log transformed"),
]

# Create log(x+1) features for crime densities
for row, (raw, log, raw_title, log_title) in enumerate(pairs):
    for col, (col_name, title, color) in enumerate([
        (raw, raw_title, '#A35139'),
        (log, log_title, '#2C3B4D')
    ]):
        ax = axes[row][col]
        ax.set_facecolor('#F7F4ED')

        ward_gdf[col_name].hist(
            bins=40, ax=ax,
            color=color, edgecolor='#F7F4ED', linewidth=0.3
        )

        ax.set_title(title, fontsize=11, fontweight='bold',
                     color='#1B2632', pad=8)
        ax.set_xlabel('crimes per km²' if col == 0 else 'log(x+1)',
                      fontsize=9, color='#1B2632')
        ax.set_ylabel('Frequency', fontsize=9, color='#1B2632')
        ax.tick_params(colors='#1B2632', labelsize=8)
        ax.spines[['top', 'right']].set_visible(False)
        ax.spines[['left', 'bottom']].set_color('#C9C1B1')
        ax.yaxis.grid(True, color='#C9C1B1', linewidth=0.5, linestyle='--')
        ax.set_axisbelow(True)

fig.suptitle('Effect of log(x+1) transformation on crime density distributions',
             fontsize=13, fontweight='bold', color='#1B2632', y=1.02)


plt.tight_layout()
plt.show()
```

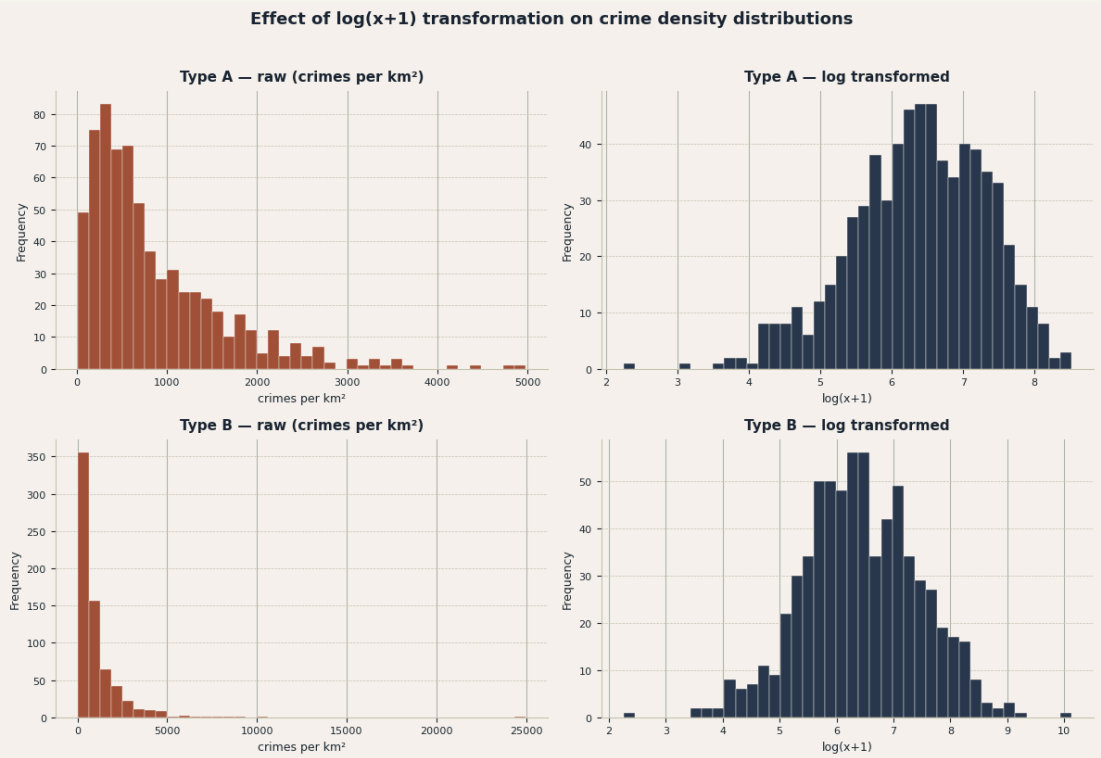


Figure 1: Effect of log(x+1) Transformation on Crime Density Distributions. *Raw (left) and log-transformed (right) distributions for Type A and Type B across 679 wards. Both raw distributions are strongly right-skewed — Type B notably more extreme (max ≈ 25,000 crimes per km²). log(x+1) produces approximately symmetric, unimodal distributions suitable for regression modelling.*

In [66]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 5), facecolor='#F7F4ED')

for ax, col, color, title in zip(
    axes,
    ["Before", "After"],
    ["#A35139", "#2C3B4D"],
    ["Before log transform", "After log transform"]
):
    ax.set_facecolor('#F7F4ED')
    bars = ax.barh(skew_df.index, skew_df[col],
                   color=color, edgecolor='none')
    ax.axvline(0, color='#1B2632', linewidth=0.8, linestyle='--')
    ax.axvline(1.5, color='#FFB162', linewidth=0.8,
```

```
                linestyle='--', label='Skewness = 1.5')
    ax.axvline(-1.5, color='#FFB162', linewidth=0.8, linestyle='--')
    ax.set_title(title, fontsize=12, fontweight='bold',
                 color='#1B2632', pad=10)
    ax.set_xlabel('Skewness', fontsize=10, color='#1B2632')
    ax.tick_params(colors='#1B2632', labelsize=9)
    ax.spines[['top', 'right']].set_visible(False)
    ax.spines[['left', 'bottom']].set_color('#C9C1B1')
    ax.xaxis.grid(True, color='#C9C1B1', linewidth=0.5, linestyle='--')
    ax.set_axisbelow(True)
    if col == "After":
        ax.set_xlim(-0.5, skew_df["After"].max() + 0.5)
    else:
        ax.set_xlim(0, skew_df["Before"].max() + 0.5)
    ax.legend(fontsize=8, framealpha=0)

fig.suptitle('Skewness before and after log(x+1) transformation',
             fontsize=13, fontweight='bold', color='#1B2632', y=1.02)

plt.tight_layout()
plt.show()
```

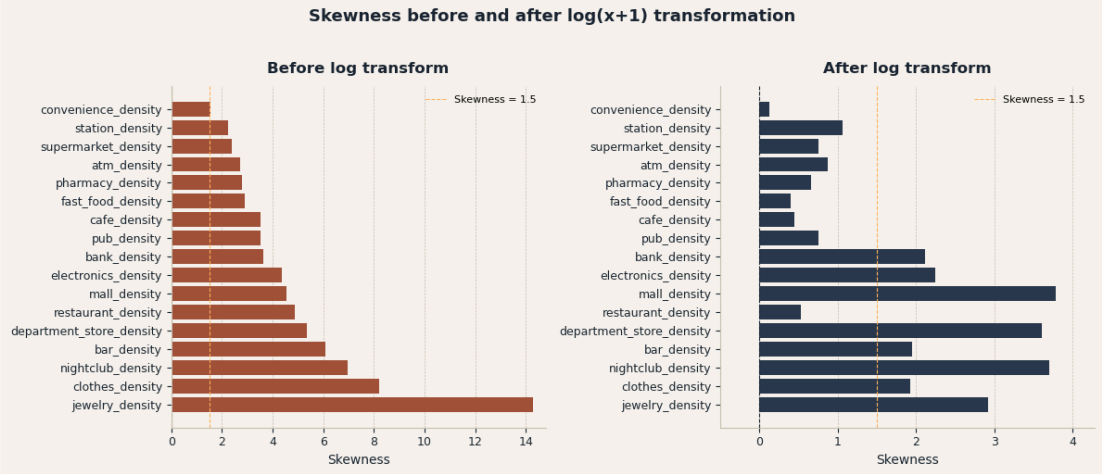


Figure 2: Skewness Before and After log(x+1) Transformation — POI Density Features *Pre-transformation, all 17 features exceed the ±1.5 threshold — jewelry_density most severely (skewness ≈ 14). Post-transformation, 12 fall within the acceptable range. The five remaining (mall, electronics, nightclub, department store, jewelry) retain elevated skew due to zero-inflation across outer-London wards, noted as a modelling limitation.*

In [67]:
```
# Save feature matrix
save_cols = (
    ["WardCode", "WardName", "Borough", "area_km2",
     "population",
     "typeA_count", "typeB_count",
     "typeA_density", "typeB_density"]
    + poi_density_cols
    + log_feature_cols
    + [log_targetA, log_targetB]
)

ward_gdf[save_cols].to_csv("data/feature_matrix.csv", index=False)
```

In [68]:
```
# Load feature matrix
feature_matrix = pd.read_csv(BASE + "feature_matrix.csv")
print(f"Feature matrix loaded — {feature_matrix.shape[0]} wards × {feature_matrix.shape[1]} columns")
# feature_matrix.head(5)
```

```
Feature matrix loaded — 679 wards × 45 columns
```

Table 6: Variable definitions, sources, units, and roles in the analysis

Variable	Type	Source	Unit	Role
typeA_density	Continuous	MPS / London Datastore	crimes per km²	Target — Type A
typeB_density	Continuous	MPS / London Datastore	crimes per km²	Target — Type B
restaurant_density	Continuous	OpenStreetMap	POIs per km²	Feature
cafe_density	Continuous	OpenStreetMap	POIs per km²	Feature
fast_food_density	Continuous	OpenStreetMap	POIs per km²	Feature
convenience_density	Continuous	OpenStreetMap	POIs per km²	Feature
clothes_density	Continuous	OpenStreetMap	POIs per km²	Feature
atm_density	Continuous	OpenStreetMap	POIs per km²	Feature

Variable	Type	Source	Unit	Role
pub_density	Continuous	OpenStreetMap	POIs per km²	Feature
pharmacy_density	Continuous	OpenStreetMap	POIs per km²	Feature
bar_density	Continuous	OpenStreetMap	POIs per km²	Feature
supermarket_density	Continuous	OpenStreetMap	POIs per km²	Feature
station_density	Continuous	OpenStreetMap	POIs per km²	Feature
bank_density	Continuous	OpenStreetMap	POIs per km²	Feature
jewelry_density	Continuous	OpenStreetMap	POIs per km²	Feature
electronics_density	Continuous	OpenStreetMap	POIs per km²	Feature
nightclub_density	Continuous	OpenStreetMap	POIs per km²	Feature
department_store_density	Continuous	OpenStreetMap	POIs per km²	Feature
mall_density	Continuous	OpenStreetMap	POIs per km²	Feature
area_km2	Continuous	ONS / EPSG:27700	km²	Normalisation denominator
population	Continuous	ONS mid-2021	persons	Context variable

Note: All log-transformed versions of the above density variables (suffix _log) are used as model inputs.

In [69]:
```
# Choropleth maps — crime and POI densities

fig, axes = plt.subplots(1, 3, figsize=(16, 6), facecolor='#F7F4ED')

cmap_typeA = LinearSegmentedColormap.from_list('typeA', ['#F7F4ED', '#A35139'])
cmap_typeB = LinearSegmentedColormap.from_list('typeB', ['#F7F4ED', '#2C3B4D'])
cmap_poi   = LinearSegmentedColormap.from_list('poi',   ['#F7F4ED', '#FFB162'])

plots = [
    ("typeA_density",     "Person & Society Crime\n(crimes per km²)", cmap_typeA, "a"),
    ("typeB_density",     "Property Crime\n(crimes per km²)",          cmap_typeB, "b"),
    ("fast_food_density", "Fast Food\n(POIs per km²)",         cmap_poi,   "c"),
]

ward_plot = ward_gdf.to_crs("EPSG:27700")
gla_plot  = gla_gdf.to_crs("EPSG:27700")

minx, miny, maxx, maxy = gla_plot.total_bounds
pad_x = (maxx - minx) * 0.02
pad_y = (maxy - miny) * 0.02

for ax, (col, title, cmap, label) in zip(axes, plots):
    ax.set_facecolor('#F7F4ED')
    vmax = ward_plot[col].quantile(0.95)

    gla_plot.plot(ax=ax, color='#D8D3C8', edgecolor='none')

    ward_plot.plot(
        column=col,
        cmap=cmap,
        legend=True,
        vmin=0,
        vmax=vmax,
        linewidth=0.1,
        edgecolor='#C9C1B1',
        legend_kwds={
            "shrink": 0.8,
            "aspect": 25,
            "pad": 0.02,
            "format": "%.0f"
        },
        ax=ax,
        missing_kwds={"color": "#C9C1B1"}
    )

    gla_plot.boundary.plot(ax=ax, color='#1B2632', linewidth=1.0)

    # Style the colorbar geopandas created
    cbar = ax.get_figure().axes[-1]
    cbar.tick_params(labelsize=8, colors='#1B2632')
    cbar.set_facecolor('#F7F4ED')
    for spine in cbar.spines.values():
        spine.set_visible(False)

    ax.set_xlim(minx - pad_x, maxx + pad_x)
    ax.set_ylim(miny - pad_y, maxy + pad_y)

    ax.set_title(title, fontsize=10, fontweight='bold',
                 color='#1B2632', pad=8)
    ax.text(0.02, 0.98, label, transform=ax.transAxes,
            fontsize=11, fontweight='bold', color='#1B2632',
```

```
            va='top', ha='left')
    ax.set_axis_off()

fig.suptitle('Crime and amenity density across London wards',
             fontsize=13, fontweight='bold', color='#1B2632', y=1.01)

plt.tight_layout()
plt.show()
```

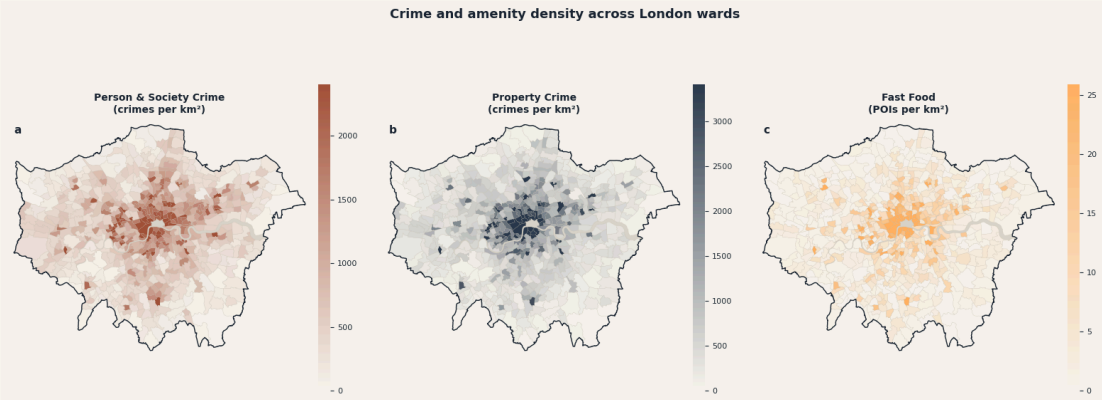


Figure 3: Spatial distribution of crime and amenity density across London wards *Both crime types show strong centre-periphery gradients — fast-food density closely mirrors this spatial pattern. Values per km², capped at the 95th percentile.*

With the feature matrix prepared — 679 wards × 17 log-transformed POI density features — four models of increasing complexity were applied to predict crime density across London wards.

Methodology

[go back to the top]

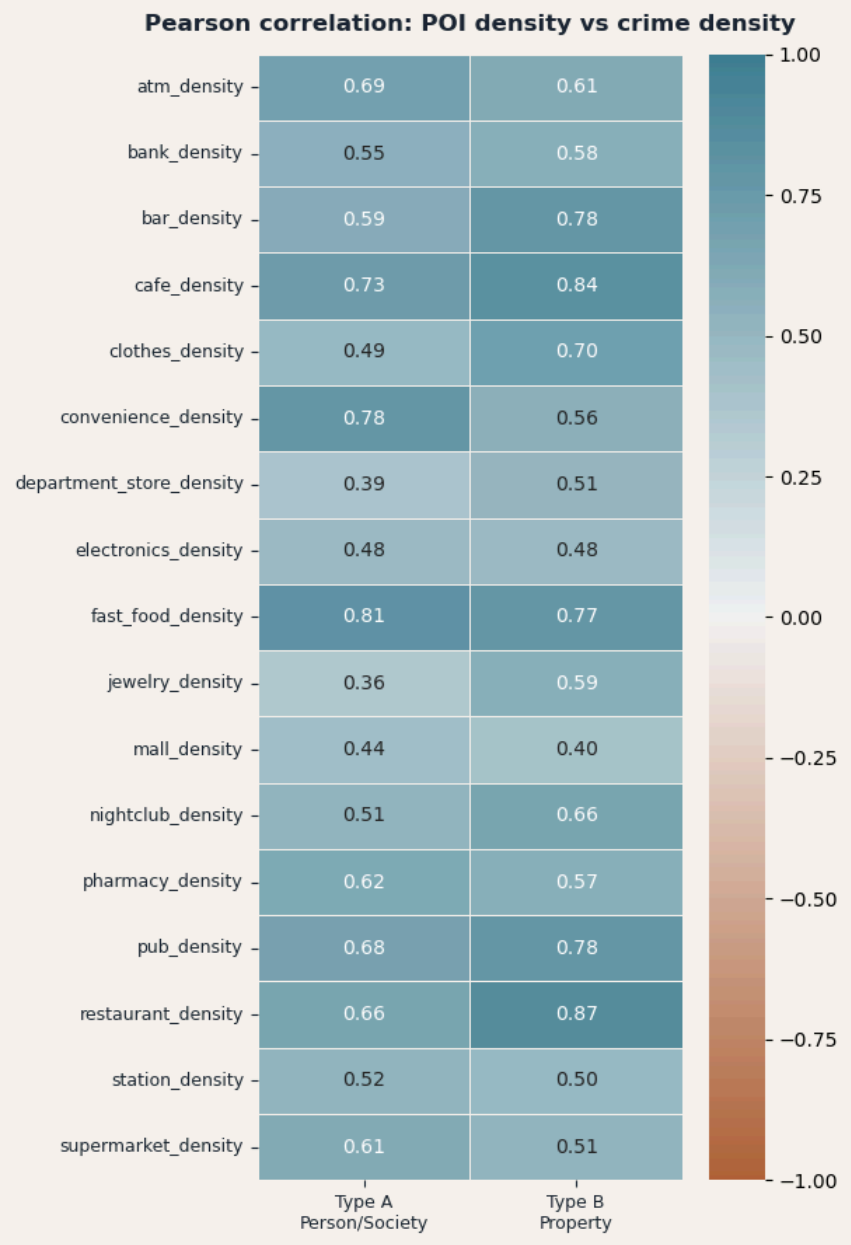
Pre-Modelling Diagnostics:

Three diagnostic checks were conducted before modelling: POI-crime correlations (Figure 4), feature multicollinearity (Figure 5), and linearity checks (Figure 6). Results confirmed strong associations, significant multicollinearity among food and beverage features, and non-linear relationships for several predictors — motivating the method sequence below.

In [70]:
```
# Pre-modelling diagnostics
# ==========================================================================================
# Figure 4:Pearson Correlation Between POI Density and Crime Density by Category
# Figure 5: Pearson Intercorrelation Matrix for POI Density Features
# Figure 6: Linearity Check – Top POI Features vs Type A Crime Density
```

In [71]:
```
# Correlation heatmap: POI density vs crime targets

# We will compute the Pearson correlation coefficients between each POI density feature and the two crime de
poi_density_cols = [c for c in feature_matrix.columns
                    if c.endswith('_density')
                    and c not in ['typeA_density', 'typeB_density']]

# We will focus on the raw density features for correlation analysis, as the log transformation is primarily
target_cols  = ["typeA_density", "typeB_density"]

# Compute correlation matrix and extract relevant correlations
corr = feature_matrix[poi_density_cols + target_cols].corr()
corr_targets = corr.loc[poi_density_cols, target_cols]

# Heatmap of correlations
fig, ax = plt.subplots(figsize=(6, 9), facecolor='#F7F4ED')
ax.set_facecolor('#F7F4ED')

sns.heatmap(
    corr_targets,
    annot=True, fmt=".2f",
    cmap=sns.diverging_palette(30, 220, as_cmap=True),
    center=0,
    vmin=-1, vmax=1,
    xticklabels=["Type A\nPerson/Society", "Type B\nProperty"],
    linewidths=0.5,
    linecolor='#F7F4ED',
    ax=ax
)

ax.set_title("Pearson correlation: POI density vs crime density",
             fontsize=12, fontweight='bold', color='#1B2632', pad=12)
ax.tick_params(colors='#1B2632', labelsize=9)

plt.tight_layout()
plt.show()
```

```
print("\nTop predictors — Type A (person/society crime):")
print(corr_targets["typeA_density"].sort_values(ascending=False).round(2).head(5))
print("\nTop predictors — Type B (property crime):")
print(corr_targets["typeB_density"].sort_values(ascending=False).round(2).head(5))
```

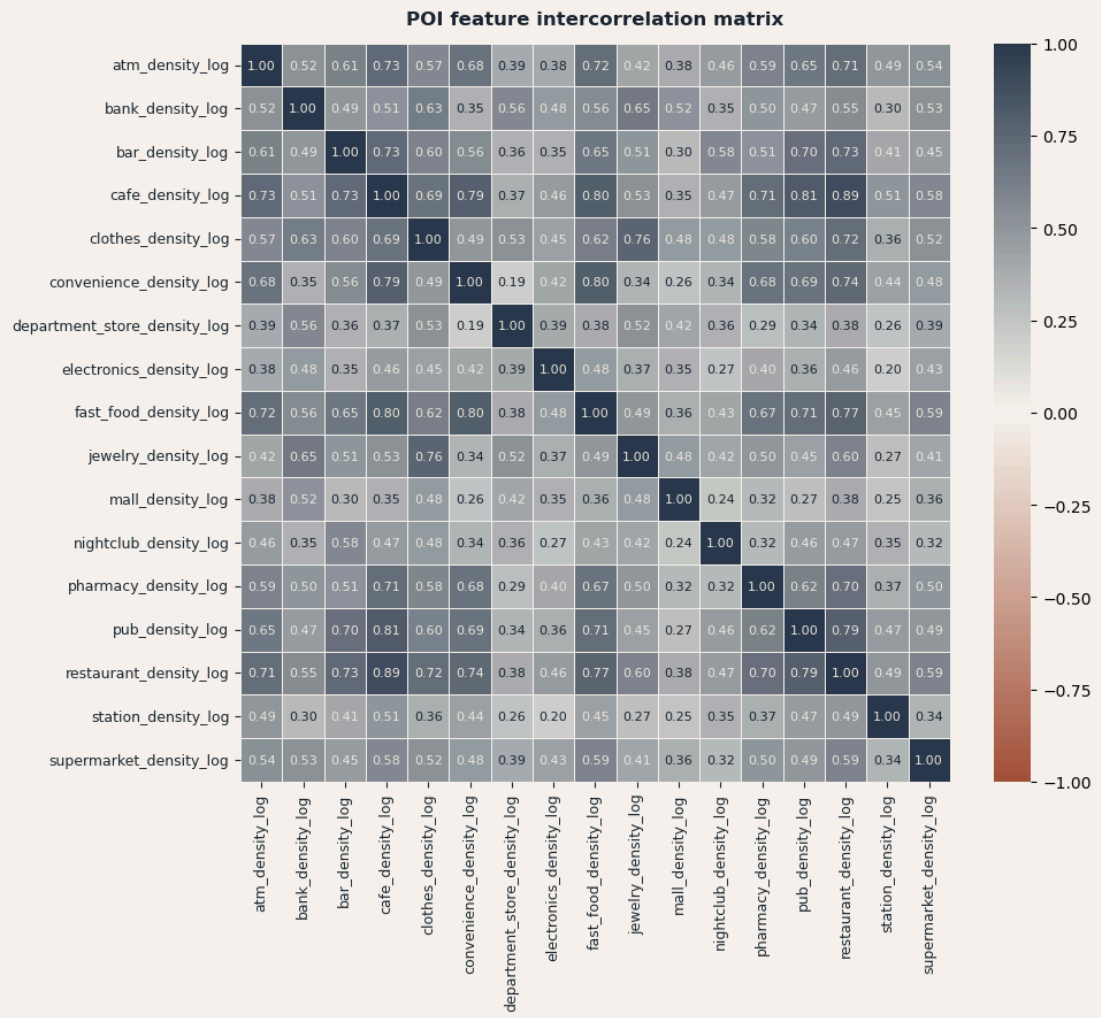



```
Top predictors — Type A (person/society crime):
fast_food_density      0.81
convenience_density    0.78
cafe_density           0.73
atm_density            0.69
pub_density            0.68
Name: typeA_density, dtype: float64

Top predictors — Type B (property crime):
restaurant_density    0.87
cafe_density          0.84
pub_density           0.78
bar_density           0.78
fast_food_density     0.77
Name: typeB_density, dtype: float64
```

Figure 4: Pearson Correlation Between POI Density and Crime Density by Category *Pearson correlation coefficients between log-transformed POI density features and crime density targets across 679 London wards. Fast food (r=0.81) and convenience store (r=0.78) are the strongest correlates of Type A crime; restaurant (r=0.87) and café (r=0.84) lead for Type B.*

In [72]:
```
# Multicollinearity check between POI features
poi_corr = feature_matrix[log_feature_cols].corr()

# Heatmap of POI feature intercorrelations with adaptive annotations
fig, ax = plt.subplots(figsize=(10, 9), facecolor='#F7F4ED')
ax.set_facecolor('#F7F4ED')

# We will compute the Pearson correlation coefficients between the log-transformed POI density features to c
div_cmap = LinearSegmentedColormap.from_list(
    'div', ['#A35139', '#F7F4ED', '#2C3B4D']
)
```

```python
# Create heatmap without annotations to allow for custom annotation with adaptive text color
im = sns.heatmap(
    poi_corr,
    annot=False,          # turn off seaborn annotations
    cmap=div_cmap, center=0,
    vmin=-1, vmax=1,
    linewidths=0.5,
    linecolor='#F7F4ED',
    ax=ax
)

# Manual annotations with adaptive colour
for i in range(len(poi_corr)):
    for j in range(len(poi_corr.columns)):
        val = poi_corr.iloc[i, j]
        # White text for dark cells, dark text for light/mid cells
        text_color = '#EEE9DF' if abs(val) > 0.4 else '#1B2632'
        ax.text(
            j + 0.5, i + 0.5, f"{val:.2f}",
            ha='center', va='center',
            fontsize=8, color=text_color
        )

ax.set_title('POI feature intercorrelation matrix',
             fontsize=12, fontweight='bold', color='#1B2632', pad=12)
ax.tick_params(colors='#1B2632', labelsize=9)


plt.tight_layout()
plt.show()

# Flag highly correlated pairs
high_corr = [
    (poi_corr.columns[i], poi_corr.columns[j], round(poi_corr.iloc[i, j], 2))
    for i in range(len(poi_corr.columns))
    for j in range(i + 1, len(poi_corr.columns))
    if abs(poi_corr.iloc[i, j]) > 0.8
]

print("Highly correlated POI pairs (r > 0.8):")
for a, b, r in high_corr:
    print(f"  {a} × {b}: r = {r}")
```




```
Highly correlated POI pairs (r > 0.8):
  cafe_density_log × fast_food_density_log: r = 0.8
  cafe_density_log × pub_density_log: r = 0.81
  cafe_density_log × restaurant_density_log: r = 0.89
```

Figure 5: POI feature intercorrelation matrix *Pairwise Pearson correlation coefficients between all 17 log-transformed POI density features. Three pairs exceed r=0.80: café × restaurant (r=0.89), café × pub*

(r=0.81), and café × fast food (r=0.80), reflecting the spatial co-occurrence of food and beverage amenities in inner-London wards. This multicollinearity violates MLR independence assumptions and justifies Random Forest as the primary model.

In [73]:
```
# Linearity check: top POI features vs crime targets
fig, axes = plt.subplots(2, 3, figsize=(14, 8), facecolor='#F7F4ED')

# We will create scatter plots of the top POI density features (based on correlation with type A crime densi
top_features = [
    "convenience_density_log", "fast_food_density_log",
    "restaurant_density_log", "cafe_density_log",
    "atm_density_log", "pub_density_log"
]

# We will plot the log-transformed POI density features against the log-transformed type A crime density to
for ax, feat in zip(axes.flatten(), top_features):
    ax.set_facecolor('#F7F4ED')

    ax.scatter(
        feature_matrix[feat],
        feature_matrix["typeA_density_log"],
        alpha=0.25, s=12, color='#A35139', edgecolors='none'
    )

    # Add lowess trend line
    sorted_x = feature_matrix[feat].sort_values()
    trend = lowess(
        feature_matrix["typeA_density_log"],
        feature_matrix[feat],
        frac=0.4
    )
    ax.plot(trend[:, 0], trend[:, 1],
            color='#2C3B4D', linewidth=1.5)

    label = feat.replace("_density_log", "").replace("_", " ")
    ax.set_xlabel(f'{label} density (log)', fontsize=9, color='#1B2632')
    ax.set_ylabel('Type A crime (log)', fontsize=9, color='#1B2632')
    ax.set_title(label.title(), fontsize=10, fontweight='bold',
                 color='#1B2632', pad=6)
    ax.tick_params(colors='#1B2632', labelsize=8)
    ax.spines[['top', 'right']].set_visible(False)
    ax.spines[['left', 'bottom']].set_color('#C9C1B1')

fig.suptitle('Linearity check: top POI features vs Type A crime density',
             fontsize=13, fontweight='bold', color='#1B2632', y=1.02)


plt.tight_layout()
plt.show()
```

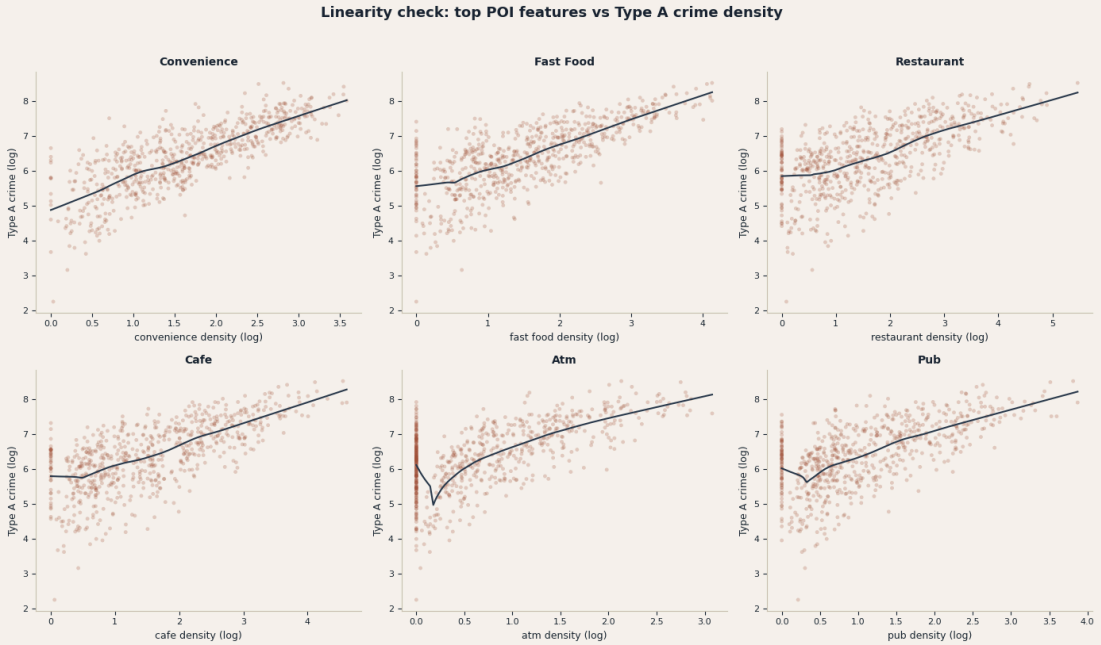
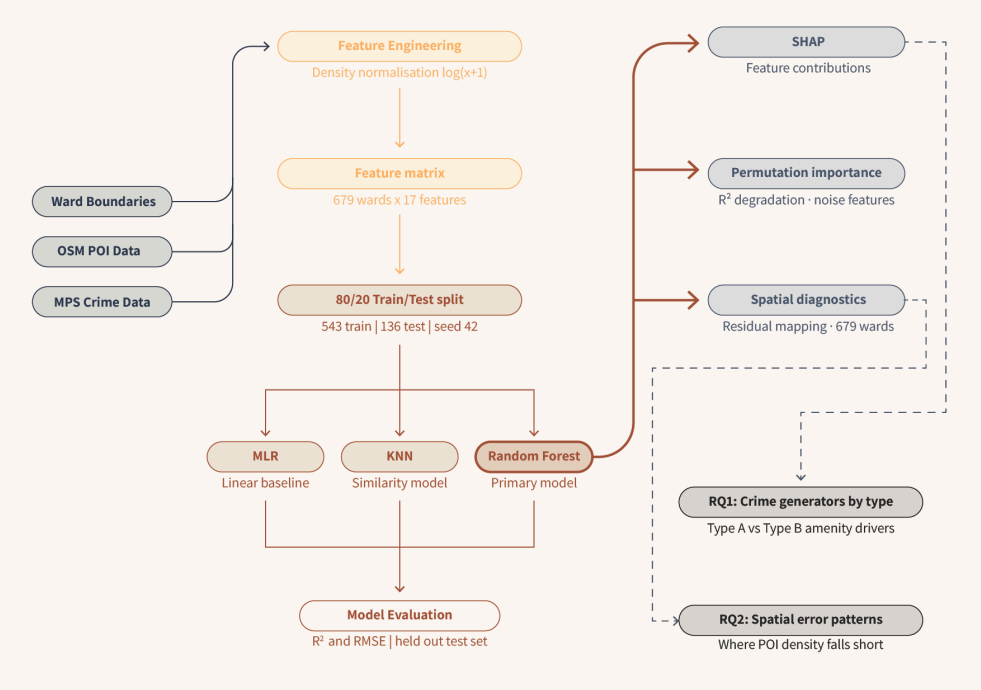


Figure 6: Linearity Check — Top POI Features vs Type A Crime Density *Scatter plots with LOWESS trend lines across 679 London wards. Convenience, fast food, restaurants, and cafés display broadly linear trends, while ATMs and pubs exhibit notable nonlinearity — flat at low densities before rising sharply. This confirms that a linear model would misrepresent several key relationships among features.*

Methodology:

In [74]:
```
# Load diagram from GitHub (raw URL)
url = "https://raw.githubusercontent.com/EmilyDeeb/0006_Assessment/main/Diagram.png"
response = requests.get(url)
```

```
img = PILImage.open(BytesIO(response.content))
display(Image(url=url, width=700))
```



> *The dataset was split 80/20 into training (543 wards) and test (136 wards) sets using a fixed random seed (42) for reproducibility. 5-fold cross-validation on the training set confirmed that the results are not sensitive to the particular split.*

Table 7: Model sequence, role, and justification for method selection

Method	Role	Justification
MLR	Linear baseline	Benchmarking only — multicollinearity and non-linearity confirmed in Figures 5 and 6 mean assumptions do not hold
KNN	Similarity model	Tests whether wards with similar amenity profiles share crime levels; k=4 selected via 5-fold CV (Figure 9)
RF	Primary model	MLR assumptions violated; KNN cannot capture interactions between amenity types; hyperparameters tuned via grid search: n_estimators=300, max_depth=10, min_samples_leaf=4
SHAP	Interpretation	Decomposes RF predictions into individual feature contributions — showing which amenity types drive each crime category, by how much, and in which direction

In [75]:
```
# Mythodology section
# Split data into training and test sets
# ==========================================================================================
# Figure 7+8: MLR Standardised Coefficients — Type A Person and Society Crime + Type B Property Crime
# Figure 9: KNN — Cross-Validated R² by k (Mean of Both Targets)
# Table 7: Model Performance on Held-Out Test Set
# Figure 10: Model Performance Comparison — R² on Held-Out Test Set
# Figure 11: SHAP Feature Contributions — Type A Person and Society Crime
# Figure 12: SHAP Feature Contributions — Type B Property Crime
```

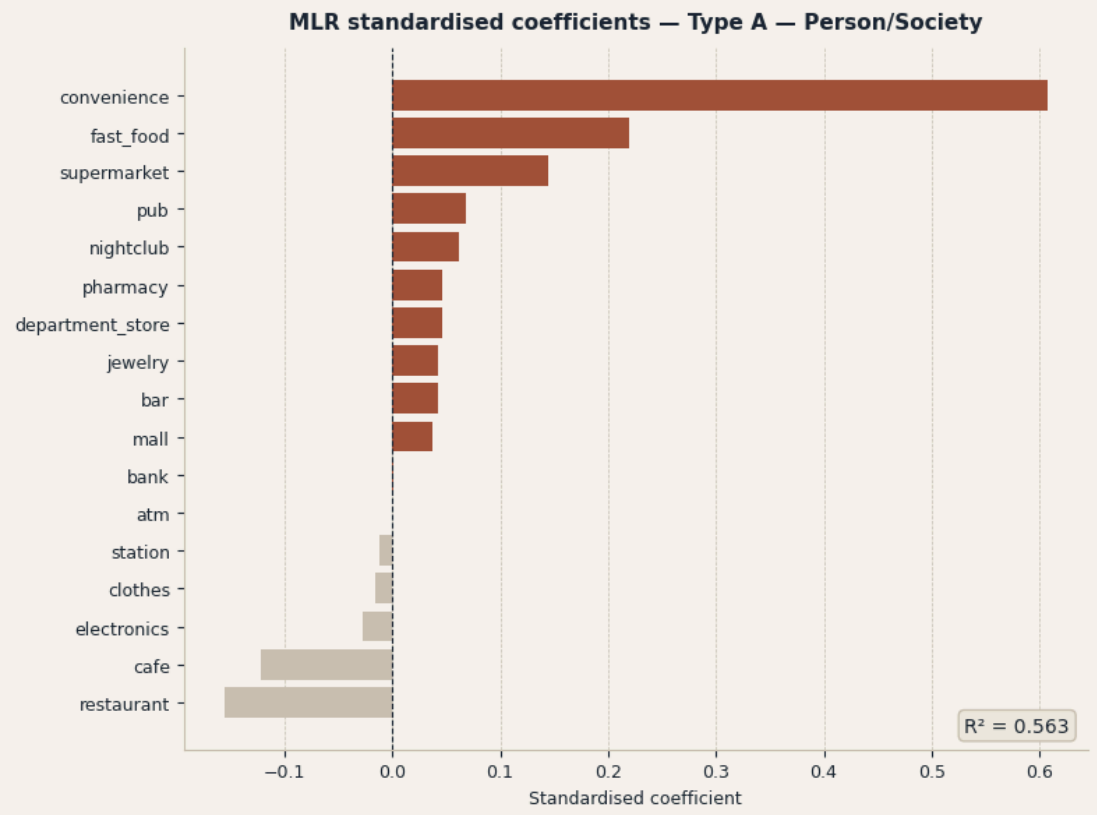
In [76]:
```
# Feature setup + train/test split
# Features and targets — drawn from feature matrix
log_feature_cols = [c for c in feature_matrix.columns
                    if c.endswith('_density_log')
                    and c not in ['typeA_density_log', 'typeB_density_log']]

feature_cols = log_feature_cols
targetA      = "typeA_density_log"
targetB      = "typeB_density_log"

# Drop any rows with missing values
model_df = feature_matrix[feature_cols + [targetA, targetB]].dropna().copy()
```

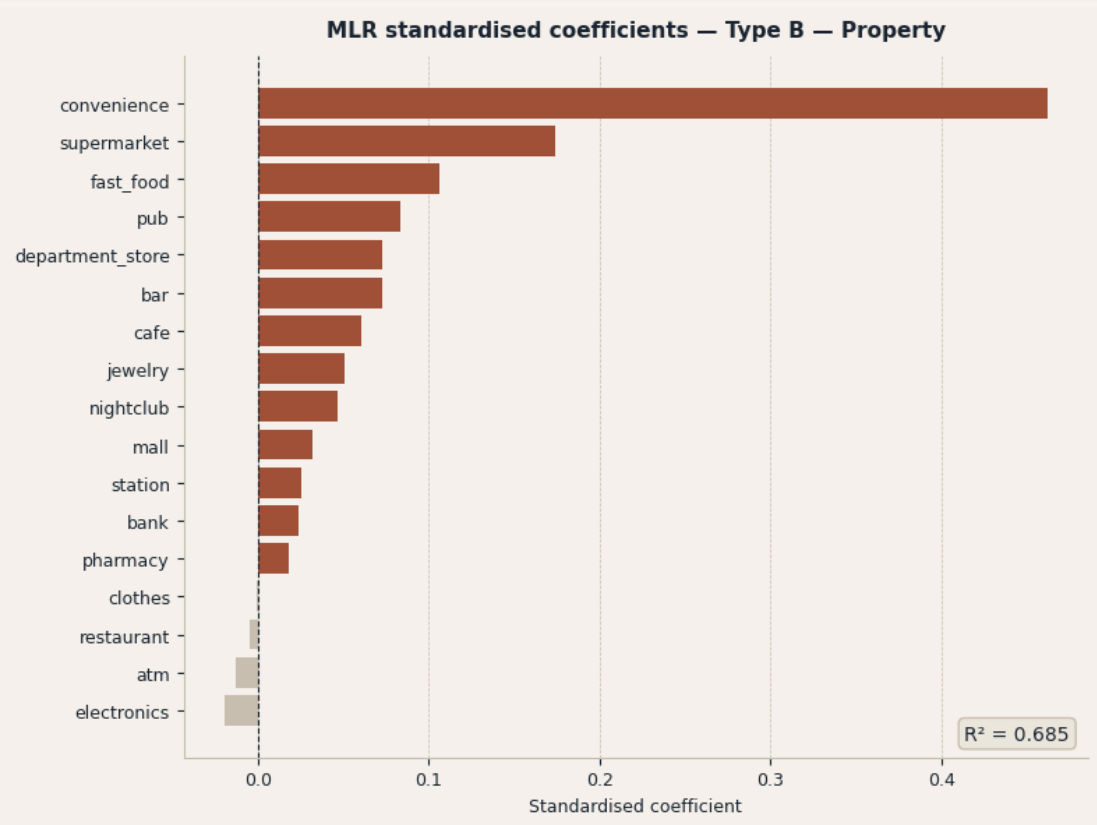
```python
X  = model_df[feature_cols]
yA = model_df[targetA]
yB = model_df[targetB]

X_train, X_test, yA_train, yA_test, yB_train, yB_test = train_test_split(
    X, yA, yB, test_size=0.2, random_state=42
)

# Standardise for MLR and KNN (RF does not require this)
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)

print(f"Modelling dataset: {len(model_df)} wards")
print(f"Train: {X_train.shape[0]} wards | Test: {X_test.shape[0]} wards")
print(f"Features: {len(feature_cols)}")
print(f"\nFeature list: {[f.replace('_density_log', '') for f in feature_cols]}")
```

```
Modelling dataset: 679 wards
Train: 543 wards | Test: 136 wards
Features: 17

Feature list: ['atm', 'bank', 'bar', 'cafe', 'clothes', 'convenience', 'department_store', 'electronics', 'f
ast_food', 'jewelry', 'mall', 'nightclub', 'pharmacy', 'pub', 'restaurant', 'station', 'supermarket']
```

In [77]:

```python
# Multiple Linear Regression

results = {}

# We will fit a multiple linear regression model to predict the log-transformed crime density targets (typeA
# using the log-transformed POI density features. We will evaluate model performance using R²
# and RMSE on the test set, and we will also visualize the standardised coefficients to understand
# which features are most influential in predicting crime densities.
for target_name, y_train, y_test in [
    ("Type A — Person/Society", yA_train, yA_test),
    ("Type B — Property",       yB_train, yB_test)
]:
    mlr = LinearRegression()
    mlr.fit(X_train_sc, y_train)
    y_pred = mlr.predict(X_test_sc)

    r2   = r2_score(y_test, y_pred)
    rmse = root_mean_squared_error(y_test, y_pred)
    results[f"MLR — {target_name}"] = {"R²": round(r2, 3), "RMSE": round(rmse, 3)}

    # Coefficient plot
    coef_df = pd.DataFrame({
        "Feature":     [c.replace("_density_log", "") for c in feature_cols],
        "Coefficient": mlr.coef_
    }).sort_values("Coefficient", ascending=True)

    fig, ax = plt.subplots(figsize=(8, 6), facecolor='#F7F4ED')
    ax.set_facecolor('#F7F4ED')

    colors = ['#A35139' if c > 0 else '#C9C1B1' for c in coef_df["Coefficient"]]
    ax.barh(coef_df["Feature"], coef_df["Coefficient"],
            color=colors, edgecolor='none')
    ax.axvline(0, color='#1B2632', linewidth=0.8, linestyle='--')

    ax.set_title(f'MLR standardised coefficients — {target_name}',
                 fontsize=11, fontweight='bold', color='#1B2632', pad=10)
    ax.set_xlabel('Standardised coefficient', fontsize=9, color='#1B2632')
    ax.tick_params(colors='#1B2632', labelsize=9)
    ax.spines[['top', 'right']].set_visible(False)
    ax.spines[['left', 'bottom']].set_color('#C9C1B1')
    ax.xaxis.grid(True, color='#C9C1B1', linewidth=0.5, linestyle='--')
    ax.set_axisbelow(True)

    # R² annotation on plot
    ax.text(0.98, 0.02, f'R² = {r2:.3f}',
            transform=ax.transAxes, ha='right', va='bottom',
            fontsize=10, color='#1B2632',
            bbox=dict(boxstyle='round,pad=0.3', facecolor='#EEE9DF',
                      edgecolor='#C9C1B1'))

    plt.tight_layout()
    plt.show()

# Summary
print("\nMLR Summary:")
for k, v in results.items():
    print(f"  {k}: R² = {v['R²']:.3f} | RMSE = {v['RMSE']:.3f}")
```






```
MLR Summary:
  MLR — Type A — Person/Society: R² = 0.563 | RMSE = 0.565
  MLR — Type B — Property: R² = 0.685 | RMSE = 0.540
```

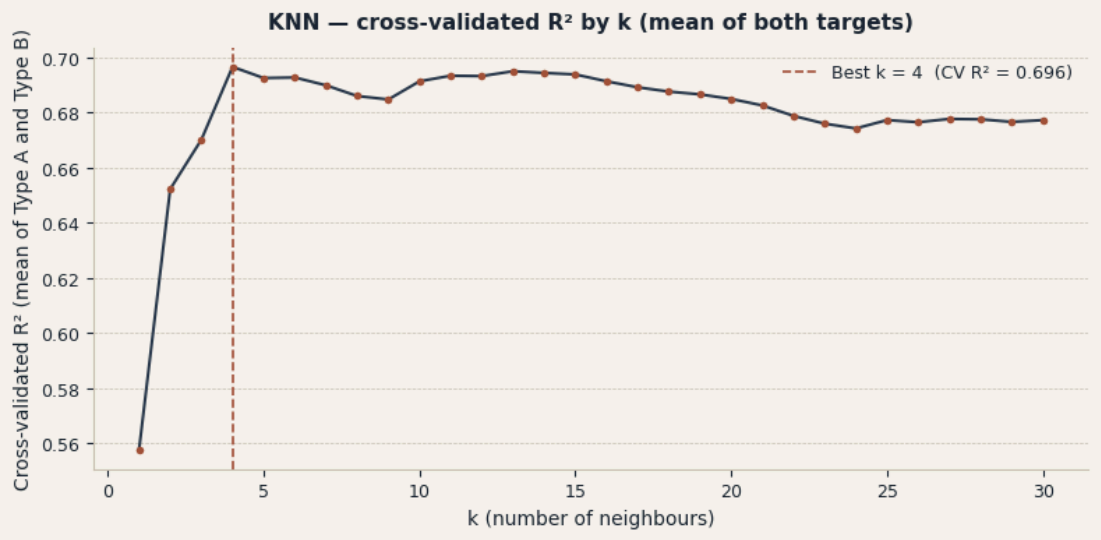
Figure 7: MLR Standardised Coefficients — Type A Person and Society Crime *Standardised coefficients for Type A crime (R^2=0.563). Convenience store is the dominant predictor. Negative coefficients for restaurant and café are multicollinearity artefacts — both show positive SHAP values in the Random Forest model.*

Figure 8: MLR Standardised Coefficients — Type B Property Crime *Standardised coefficients for Type B crime (R^2=0.685). Convenience store again dominates, with supermarket and fast food as secondary predictors. Negative coefficients for restaurant and ATM reflect multicollinearity rather than genuine inverse relationships.*

In [78]:

```
#  KNN Regressor (tune k)
# Tune k using average of Type A and Type B CV R²
k_scores = []
for k in range(1, 31):
    knn = KNeighborsRegressor(n_neighbors=k)
    scoreA = cross_val_score(knn, X_train_sc, yA_train, cv=5, scoring="r2").mean()
```

```
    scoreB = cross_val_score(knn, X_train_sc, yB_train, cv=5, scoring="r2").mean()
    k_scores.append((scoreA + scoreB) / 2)

best_k = k_scores.index(max(k_scores)) + 1

# K tuning plot
fig, ax = plt.subplots(figsize=(8, 4), facecolor='#F7F4ED')
ax.set_facecolor('#F7F4ED')

ax.plot(range(1, 31), k_scores,
        color='#2C3B4D', linewidth=1.5, marker='o',
        markersize=4, markerfacecolor='#A35139', markeredgewidth=0)
ax.axvline(best_k, color='#A35139', linestyle='--', linewidth=1.2,
           label=f'Best k = {best_k}  (CV R² = {max(k_scores):.3f})')

ax.set_xlabel('k (number of neighbours)', fontsize=10, color='#1B2632')
ax.set_ylabel('Cross-validated R² (mean of Type A and Type B)', fontsize=10, color='#1B2632')
ax.set_title('KNN — cross-validated R² by k (mean of both targets)',
             fontsize=11, fontweight='bold', color='#1B2632', pad=10)
ax.tick_params(colors='#1B2632', labelsize=9)
ax.spines[['top', 'right']].set_visible(False)
ax.spines[['left', 'bottom']].set_color('#C9C1B1')
ax.yaxis.grid(True, color='#C9C1B1', linewidth=0.5, linestyle='--')
ax.set_axisbelow(True)
ax.legend(fontsize=9, framealpha=0, labelcolor='#1B2632')

plt.tight_layout()
plt.show()

# Fit KNN with best k — both targets
for target_name, y_train, y_test in [
    ("Type A — Person/Society", yA_train, yA_test),
    ("Type B — Property",       yB_train, yB_test)
]:
    knn = KNeighborsRegressor(n_neighbors=best_k)
    knn.fit(X_train_sc, y_train)
    y_pred = knn.predict(X_test_sc)

    r2   = r2_score(y_test, y_pred)
    rmse = root_mean_squared_error(y_test, y_pred)

    results[f"KNN — {target_name}"] = {"R²": round(r2, 3), "RMSE": round(rmse, 3)}

print(f"\nKNN Summary (k={best_k}):")
for k, v in {k: v for k, v in results.items() if 'KNN' in k}.items():
    print(f"  {k}: R² = {v['R²']:.3f} | RMSE = {v['RMSE']:.3f}")
```




```
KNN Summary (k=4):
  KNN — Type A — Person/Society: R² = 0.458 | RMSE = 0.629
  KNN — Type B — Property: R² = 0.651 | RMSE = 0.568
```

Figure 9: KNN — Cross-Validated R^2 by k (Mean of Both Targets) *Cross-validated R^2 (mean of Type A and Type B) across k=1 to k=30. Performance peaks at k=4 (CV R^2=0.696) before plateauing and gradually declining. k=4 was selected for both models.*

Results and Discussion

[go back to the top]

Model performance

As shown in Table 8, Random Forest outperformed both baseline methods across all targets and metrics. Figures 7 and 8 illustrate the multicollinearity artefacts in MLR coefficients — a direct consequence of the feature intercorrelations confirmed in Figure 5 — indicating that the POI-crime relationship is non-linear and cannot be adequately captured by additive linear models. The underperformance of KNN (Table 8

and Figure 9) suggests that wards with similar POI profiles do not always exhibit similar crime levels, pointing to spatial heterogeneity in crime determinants across London's diverse ward typologies. Property crime (Type B) was predicted more accurately than Type A across all three methods, consistent with property crime being more directly opportunity-driven and therefore more strongly determined by the physical configuration of targets.

In [79]:
```
#  Random Forest (tuned) + model comparison

# We will fit a Random Forest regressor to predict the log-transformed crime density targets using the log-t
best_params = {
    "max_depth":        10,
    "min_samples_leaf": 4,
    "n_estimators":     300,
    "random_state":     42,
    "n_jobs":           -1
}

# Fit RF with best hyperparameters — both targets
rf_models = {}

# We will train separate Random Forest models for each crime density target (Type A and Type B) using the sa
for target_name, y_train, y_test in [
    ("Type A — Person/Society", yA_train, yA_test),
    ("Type B — Property",       yB_train, yB_test)
]:
    rf = RandomForestRegressor(**best_params)
    rf.fit(X_train, y_train)
    y_pred = rf.predict(X_test)

    r2   = r2_score(y_test, y_pred)
    rmse = root_mean_squared_error(y_test, y_pred)

    results[f"RF — {target_name}"] = {"R²": round(r2, 3), "RMSE": round(rmse, 3)}
    rf_models[target_name] = rf

# Model comparison table
comparison_rows = []
for key, vals in results.items():
    parts = key.split(" — ", 1)
    comparison_rows.append({
        "Method":  parts[0],
        "Target":  parts[1],
        "R²":      vals["R²"],
        "RMSE":    vals["RMSE"]
    })

comparison_df = pd.DataFrame(comparison_rows).sort_values(
    ["Target", "R²"], ascending=[True, False]
).reset_index(drop=True)

# Styled comparison table
(comparison_df.style
    .format({'R²': '{:.3f}', 'RMSE': '{:.3f}'})
    .highlight_max(subset=['R²'], color='#C9E8D1')
    .highlight_min(subset=['RMSE'], color='#C9E8D1')
    .set_properties(**{
        'font-family': 'Inter, sans-serif',
        'font-size': '13px',
        'color': '#1B2632',
        'border-bottom': '1px solid #C9C1B1',
        'padding': '8px 14px'
    })
    .set_table_styles([
        {
        "selector": "",
        "props": [
            ("margin-left", "auto"),
            ("margin-right", "auto"),
        ],
    },
        {'selector': 'thead tr',
         'props': [('background-color', '#2C3B4D'),
                   ('color', '#EEE9DF'),
                   ('font-weight', '500')]},
        {'selector': 'caption',
         'props': [('font-size', '11px'),
                   ('color', '#C9C1B1'),
                   ('font-style', 'italic'),
                   ('text-align', 'left'),
                   ('padding', '6px 0')]}
    ])
    .hide(axis='index')
)
```

Out[79]:

Method	Target	R^2	RMSE
RF	Type A — Person/Society	0.623	0.524
MLR	Type A — Person/Society	0.563	0.565
KNN	Type A — Person/Society	0.458	0.629
RF	Type B — Property	0.716	0.512
MLR	Type B — Property	0.685	0.540
KNN	Type B — Property	0.651	0.568

Table 8: Model Performance on Held-Out Test Set *R^2 and RMSE for all three models across both crime density targets (136 test wards). Random Forest outperforms both baselines across all metrics.*

In [80]:

```
# RF CV validation
# 5-fold CV on the full dataset to validate that the 80/20 test-set R²
# is not an artefact of a fortunate split.

cv_results = {}

for target_name, y_col in [
    ("Type A — Person/Society", "typeA_density_log"),
    ("Type B — Property",       "typeB_density_log"),
]:
    y = feature_matrix[y_col].values
    X = feature_matrix[feature_cols].fillna(0).values

    rf_cv = RandomForestRegressor(**best_params)
    scores = cross_val_score(rf_cv, X, y, cv=5, scoring="r2", n_jobs=-1)

    cv_results[target_name] = {
        "CV R² (mean)":  round(scores.mean(), 3),
        "CV R² (std)":   round(scores.std(),  3),
        "Test R²":       results[f"RF — {target_name}"]["R²"],
    }
    # print(f"{target_name}")
    # print(f"  CV R²:   {scores.mean():.3f} ± {scores.std():.3f}  (folds: {np.round(scores, 3)})")
    # print(f"  Test R²: {cv_results[target_name]['Test R²']}\n")

# Styled summary table
cv_df = pd.DataFrame(cv_results).T.reset_index().rename(columns={"index": "Target"})

(cv_df.style
    .format({"CV R² (mean)": "{:.3f}", "CV R² (std)": "{:.3f}", "Test R²": "{:.3f}"})
    .set_properties(**{
        "font-family": "Inter, sans-serif",
        "font-size":   "13px",
        "color":       "#1B2632",
        "border-bottom": "1px solid #C9C1B1",
        "padding":     "8px 14px",
    })
    .set_table_styles([
        {
        "selector": "",
        "props": [
            ("margin-left", "auto"),
            ("margin-right", "auto"),
        ],
    },
        {"selector": "thead tr",
         "props": [("background-color", "#2C3B4D"),
                   ("color", "#EEE9DF"),
                   ("font-weight", "500")]},
        {"selector": "caption",
         "props": [("font-size", "11px"),
                   ("color", "#C9C1B1"),
                   ("font-style", "italic"),
                   ("text-align", "left"),
                   ("padding", "6px 0")]},
    ])
    .hide(axis="index")
)
```

Out[80]:

Target	CV R^2 (mean)	CV R^2 (std)	Test R^2
Type A — Person/Society	0.677	0.064	0.623
Type B — Property	0.753	0.084	0.716

Table 9: RF 5-fold cross-validated R^2 versus held-out test R^2 for both targets. CV R^2 *RF 5-fold cross-validated R^2 versus held-out test R^2. CV R^2 exceeds test R^2 in both cases, confirming the test set result is not a product of a favourable split. The higher standard deviation in Type B reflects one difficult fold rather than systematic instability.*

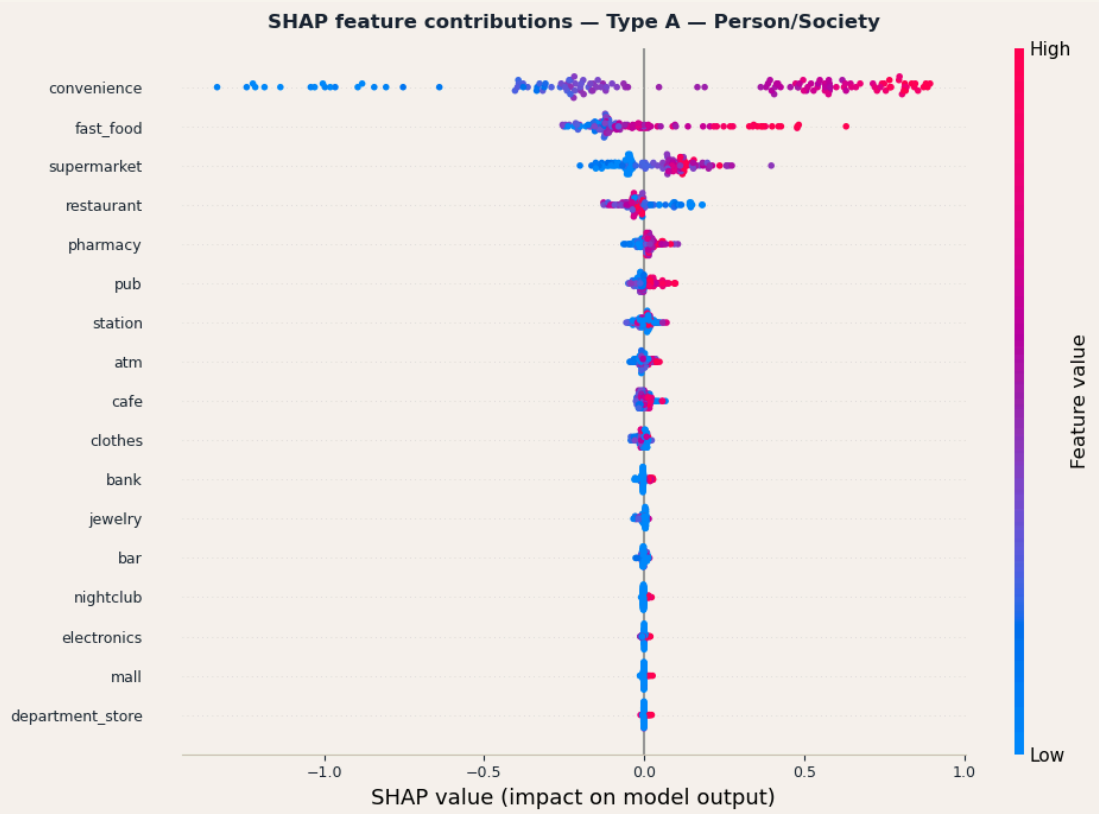
Crime generators by type

Figures 10 and 11 show the SHAP beeswarm plots for Type A and Type B, respectively.

For Type A (person/society crime), convenience store density is the dominant predictor (maximum SHAP value: 0.75), followed by fast food and supermarket density. Traditional night-time economy venues — bars, pubs, and nightclubs — were among the weakest predictors despite their prominence in criminological discourse, likely reflecting the mandatory guardianship of licensed premises — door staff, CCTV, and licensing conditions — which suppresses crime opportunity despite high footfall, consistent with Routine Activity Theory (Cohen and Felson, 1979).

For Type B (property crime), café density was the primary predictor (maximum SHAP value: 1.25), followed by convenience stores and restaurants. Fast food outlets are more strongly associated with areas where person and society crime concentrates, while cafés are more strongly associated with areas where property crime is prevalent — suggesting that POI type, not just density, determines which crime category an area generates.

In [81]:
```
#  SHAP beeswarm plots

clean_names = [c.replace("_density_log", "") for c in feature_cols]

for target_name, rf in rf_models.items():
    explainer  = shap.TreeExplainer(rf)
    shap_vals  = explainer.shap_values(X_test)

    fig, ax = plt.subplots(figsize=(10, 7), facecolor='#F7F4ED')

    shap.summary_plot(
        shap_vals,
        X_test,
        feature_names=clean_names,
        show=False,
        plot_size=None
    )

    # Style the current figure
    fig = plt.gcf()
    fig.patch.set_facecolor('#F7F4ED')
    ax = plt.gca()
    ax.set_facecolor('#F7F4ED')
    ax.tick_params(colors='#1B2632', labelsize=9)
    ax.spines[['top', 'right']].set_visible(False)
    ax.spines[['left', 'bottom']].set_color('#C9C1B1')

    plt.title(f'SHAP feature contributions — {target_name}',
              fontsize=12, fontweight='bold', color='#1B2632', pad=12)


    plt.tight_layout()
    plt.show()
    plt.close()
```



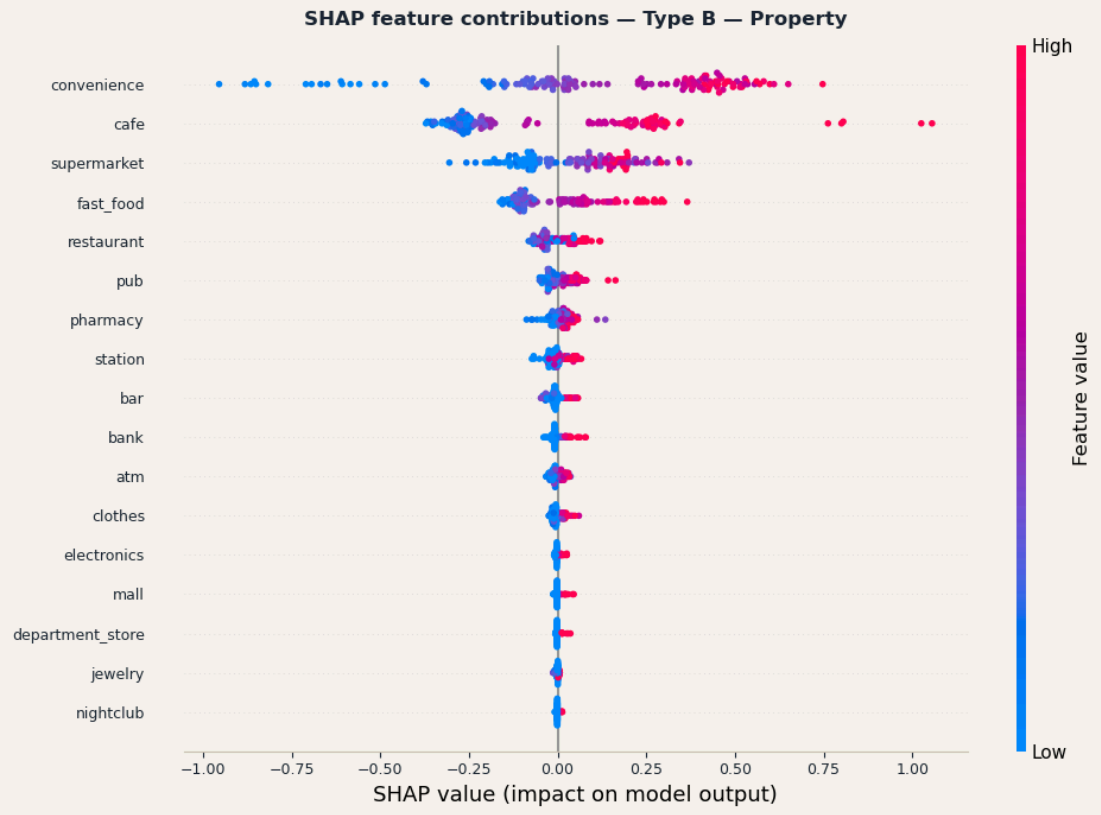


Figure 10: SHAP Feature Contributions — Type A Person and Society Crime *SHAP beeswarm plot for the Random Forest model predicting Type A crime density. Each point represents one ward; colour indicates feature value (red = high, blue = low). Convenience store density is the dominant predictor — high-density wards push predictions strongly positive, while low-density wards extend far left. Fast food and supermarket follow. Night-time economy venues (bars, nightclubs) cluster near zero, confirming their weak predictive contribution.*

Figure 11: SHAP Feature Contributions — Type B Property Crime *SHAP beeswarm plot for Type B property crime. Convenience store remains the top predictor, but café rises to second — consistent with its status as the strongest Pearson correlate of property crime (r=0.84). Café shows a pronounced negative tail, indicating low-café wards are actively suppressed in predicted property crime density. Supermarket and fast food follow.*

In [82]:

```
perm_result_A = permutation_importance(
    rf_models['Type A — Person/Society'],
    X_test,
    feature_matrix.loc[X_test.index, 'typeA_density_log'],
    scoring='r2',
    n_repeats=10,
    random_state=42,
    n_jobs=-1
)

perm_result_B = permutation_importance(
    rf_models['Type B — Property'],
    X_test,
    feature_matrix.loc[X_test.index, 'typeB_density_log'],
    scoring='r2',
    n_repeats=10,
    random_state=42,
    n_jobs=-1
)

perm_df_merged = pd.DataFrame({
    'Feature': clean_names,
    'Type A importance': perm_result_A.importances_mean.round(4),
    'Type A signal': ['Strong' if x > 0.01 else ('Weak' if x > 0 else 'Drop candidate')
                      for x in perm_result_A.importances_mean],
    'Type B importance': perm_result_B.importances_mean.round(4),
    'Type B signal': ['Strong' if x > 0.01 else ('Weak' if x > 0 else 'Drop candidate')
                      for x in perm_result_B.importances_mean],
}).sort_values('Type A importance', ascending=False).reset_index(drop=True)

(perm_df_merged.style
    .format({
        'Type A importance': '{:.4f}',
        'Type B importance': '{:.4f}'
    })
    .set_properties(**{
        'font-family': 'Inter, sans-serif',
        'font-size': '12px',
        'color': '#1B2632',
```

```
        'border-bottom': '1px solid #C9C1B1',
        'padding': '7px 12px'
    })
    .set_table_styles([
        {
        "selector": "",
        "props": [
            ("margin-left", "auto"),
            ("margin-right", "auto"),
        ],
    },
        {'selector': 'thead tr',
         'props': [('background-color', '#2C3B4D'),
                   ('color', '#EEE9DF'),
                   ('font-weight', '500')]}
    ])
    .hide(axis='index')
)
```

Out[82]:

Feature	Type A importance	Type A signal	Type B importance	Type B signal
convenience	0.6740	Strong	0.2705	Strong
fast_food	0.0542	Strong	0.0122	Strong
supermarket	0.0449	Strong	0.0918	Strong
restaurant	0.0316	Strong	0.0063	Weak
pharmacy	0.0170	Strong	0.0141	Strong
pub	0.0119	Strong	0.0087	Weak
station	0.0065	Weak	0.0005	Weak
jewelry	0.0050	Weak	0.0000	Weak
clothes	0.0030	Weak	0.0033	Weak
cafe	0.0006	Weak	0.1276	Strong
bar	0.0005	Weak	0.0016	Weak
electronics	0.0005	Weak	0.0004	Weak
nightclub	0.0002	Weak	-0.0003	Drop candidate
mall	-0.0004	Drop candidate	0.0019	Weak
department_store	-0.0007	Drop candidate	0.0008	Weak
atm	-0.0017	Drop candidate	0.0010	Weak
bank	-0.0028	Drop candidate	0.0033	Weak

Table 10: Permutation importance for Type A and Type B crime *mean R^2 degradation when each feature is shuffled across 10 repeats. Features with negative importance for a given crime type add noise and are identified as drop candidates.*

While SHAP reveals which features drive predictions, it cannot confirm whether these relationships hold independently of unmeasured variables. Permutation importance addresses this directly by measuring R^2 degradation when each feature is shuffled, and confirms the SHAP rankings across both crime types (Table 10). Convenience store density is the dominant predictor for Type A — its removal degrades R^2 by 0.674, more than ten times the next strongest feature. For Type B, café density emerges as the second strongest predictor (0.1276), providing independent confirmation that café density drives property crime distinctly from its weak contribution to Type A (0.0006). Night-time economy venues return weak or negative importance across both targets, consistent with the guardianship argument above.

Spatial diagnostics

The Random Forest explains approximately 62% of the variance in Type A crime density. To understand where the remaining error concentrates, residuals were mapped across all 679 wards (Figures 12 and 13). Underpredicted wards — where actual crime exceeds model estimates — concentrate in outer east and west London, areas characterised by high activity and footfall that POI density alone does not fully capture. Kensington and Chelsea is a notable exception, where underprediction likely reflects high-value tourist targets concentrated in a small geographic area. Overpredicted wards cluster in outer south-west London, where high amenity density does not translate to elevated crime — suggesting that the presence of amenities is a necessary but not sufficient condition for crime generation.

In [83]:

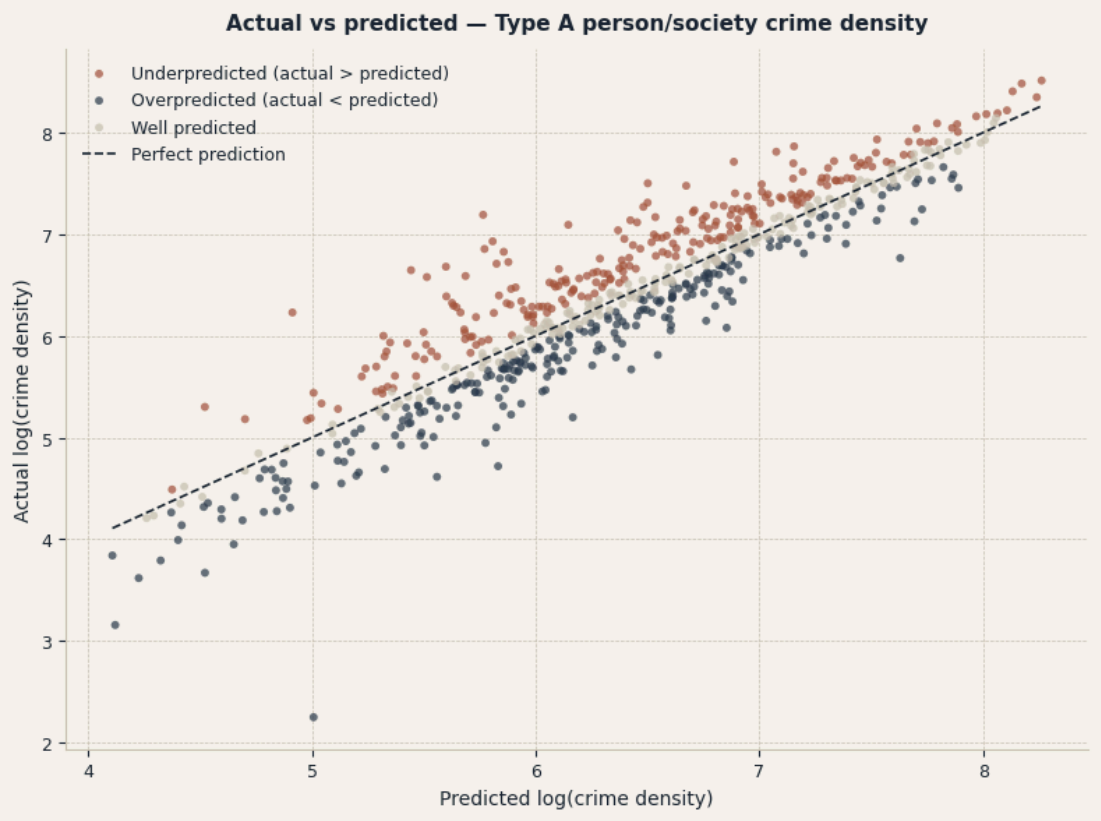
```
# Interactive actual vs predicted scatter

# Create pred_df from feature_matrix
pred_df = feature_matrix[["WardCode"] + feature_cols].copy()
pred_df["typeA_pred_log"] = rf_models["Type A – Person/Society"].predict(
    pred_df[feature_cols].fillna(0)
```

```
)
pred_df["typeB_pred_log"] = rf_models["Type B — Property"].predict(
    pred_df[feature_cols].fillna(0)
)
pred_df["residual_A"] = feature_matrix["typeA_density_log"].values - pred_df["typeA_pred_log"]
pred_df["residual_B"] = feature_matrix["typeB_density_log"].values - pred_df["typeB_pred_log"]

# Merge ward names
scatter_df = pred_df.merge(
    feature_matrix[["WardCode", "WardName", "Borough"]],
    on="WardCode", how="left"
)
scatter_df["typeA_actual"] = feature_matrix["typeA_density_log"].values
scatter_df["residual_A"]   = scatter_df["typeA_actual"] - scatter_df["typeA_pred_log"]
```

In [84]:

```
# ── Build scatter_df ──────────────────────────────────────
scatter_df = pred_df.merge(
    feature_matrix[["WardCode", "WardName", "Borough"]],
    on="WardCode", how="left"
)
scatter_df["typeA_actual"] = feature_matrix["typeA_density_log"].values
scatter_df["residual_A"]   = scatter_df["typeA_actual"] - scatter_df["typeA_pred_log"]

scatter_df["colour"] = scatter_df["residual_A"].apply(
    lambda x: '#A35139' if x > 0.1 else '#2C3B4D' if x < -0.1 else '#C9C1B1'
)
scatter_df["label"] = scatter_df["residual_A"].apply(
    lambda x: 'Underpredicted' if x > 0.1 else 'Overpredicted' if x < -0.1 else 'Well predicted'
)

# ── Figure 13 — Actual vs Predicted scatter ──────────────
fig, ax = plt.subplots(figsize=(8, 6), facecolor='#F7F4ED')
ax.set_facecolor('#F7F4ED')

for group, color, name in [
    ('Underpredicted', '#A35139', 'Underpredicted (actual > predicted)'),
    ('Overpredicted',  '#2C3B4D', 'Overpredicted (actual < predicted)'),
    ('Well predicted', '#C9C1B1', 'Well predicted'),
]:
    subset = scatter_df[scatter_df['label'] == group]
    ax.scatter(
        subset['typeA_pred_log'],
        subset['typeA_actual'],
        c=color, s=18, alpha=0.7,
        label=name, edgecolors='none'
    )

mn = scatter_df['typeA_pred_log'].min()
mx = scatter_df['typeA_pred_log'].max()
ax.plot([mn, mx], [mn, mx],
        color='#1B2632', linewidth=1.2,
        linestyle='--', label='Perfect prediction')

ax.set_xlabel('Predicted log(crime density)', fontsize=10, color='#1B2632')
ax.set_ylabel('Actual log(crime density)',    fontsize=10, color='#1B2632')
ax.set_title('Actual vs predicted — Type A person/society crime density',
             fontsize=11, fontweight='bold', color='#1B2632', pad=10)
ax.tick_params(colors='#1B2632', labelsize=9)
ax.spines[['top', 'right']].set_visible(False)
ax.spines[['left', 'bottom']].set_color('#C9C1B1')
ax.xaxis.grid(True, color='#C9C1B1', linewidth=0.5, linestyle='--')
ax.yaxis.grid(True, color='#C9C1B1', linewidth=0.5, linestyle='--')
ax.set_axisbelow(True)
ax.legend(fontsize=9, framealpha=0, labelcolor='#1B2632')

plt.tight_layout()
plt.show()

# ── Summary table ────────────────────────────────────────
summary = (
    scatter_df.groupby("label")
    .agg(
        Wards        = ("WardCode",   "count"),
        Mean_Residual= ("residual_A", "mean"),
        Max_Abs      = ("residual_A", lambda x: x.abs().max())
    )
    .round(3)
    .reset_index()
    .rename(columns={
        "label":         "Prediction Group",
        "Mean_Residual": "Mean Residual",
        "Max_Abs":       "Max |Residual|"
    })
    .sort_values("Wards", ascending=False)
)
summary["% of Wards"] = (
    summary["Wards"] / summary["Wards"].sum() * 100
).round(1).astype(str) + "%"

row_colors = {
    'Underpredicted': '#F0D5CC',
    'Overpredicted':  '#C9D4DE',
    'Well predicted': '#E8E4DC',
}
```

```python
fig2, ax2 = plt.subplots(figsize=(8, 2), facecolor='#F7F4ED')
ax2.set_facecolor('#F7F4ED')
ax2.axis('off')

cols = list(summary.columns)
cell_text = summary.values.tolist()
cell_colors = [
    [row_colors.get(row[0], '#F7F4ED')] * len(cols)
    for row in cell_text
]

table = ax2.table(
    cellText=cell_text,
    colLabels=cols,
    cellLoc='center',
    loc='center',
    cellColours=cell_colors
)
table.auto_set_font_size(False)
table.set_fontsize(10)
table.scale(1, 1.6)

for (row, col), cell in table.get_celld().items():
    cell.set_edgecolor('#C9C1B1')
    cell.set_linewidth(0.5)
    if row == 0:
        cell.set_facecolor('#2C3B4D')
        cell.get_text().set_color('#EEE9DF')
        cell.get_text().set_fontweight('bold')
    else:
        cell.get_text().set_color('#1B2632')

plt.tight_layout()
plt.show()
```



Prediction Group	Wards	Mean Residual	Max |Residual|	% of Wards
Overpredicted	252	-0.311	2.759	37.1%
Underpredicted	238	0.357	1.426	35.1%
Well predicted	189	-0.009	0.099	27.8%

Figure 12: Actual vs Predicted — Type A Person and Society Crime Density *Scatter plot of actual against predicted log crime density for all 679 wards, coloured by residual group. Overall fit is strong — 27.8% of wards are well predicted (|residual| ≤ 0.1); 37.1% overpredicted and 35.1% underpredicted. Overpredicted wards (blue, mean residual −0.311) cluster at lower predicted values, suggesting the model overestimates crime in low-density outer wards. Underpredicted wards (rust, mean residual +0.357) are more evenly distributed, indicating crime driven by factors beyond POI density.*

In [85]:
```
worst_over = scatter_df.nsmallest(1, "residual_A")[["WardName", "Borough", "residual_A"]]
print(worst_over)
```

```
    WardName  Borough  residual_A
577   Darwin  Bromley   -2.759068
```

In [86]:
```
# ── Build map_gdf ──────────────────────────────────────────────
map_gdf = ward_gdf[["WardCode", "WardName", "Borough", "geometry"]].copy()
map_gdf = map_gdf.merge(
    feature_matrix[["WardCode", "typeA_density_log"]],
    on="WardCode", how="left"
)
map_gdf["typeA_pred_log"] = rf_models["Type A — Person/Society"].predict(
    feature_matrix[feature_cols].fillna(0)
)
map_gdf["residual_A"] = (
    map_gdf["typeA_density_log"] - map_gdf["typeA_pred_log"]
).round(3)

# Remove Aviation Security artefact
map_gdf = map_gdf[
    ~map_gdf["Borough"].str.contains('Aviation|SO18', case=False, na=False)
].copy()

map_gdf = map_gdf.to_crs("EPSG:27700")
gla_plot = gla_gdf.to_crs("EPSG:27700")

# Colormap and norm
from matplotlib.colors import TwoSlopeNorm, LinearSegmentedColormap
div_cmap = LinearSegmentedColormap.from_list(
    'residual', ['#2C3B4D', '#F7F4ED', '#A35139']
)
vmax = map_gdf["residual_A"].abs().quantile(0.95)
norm = TwoSlopeNorm(vmin=-vmax, vcenter=0, vmax=vmax)
```

In [87]:
```
# ── Borough boundaries ─────────────────────────────────────────
# Dissolve wards to borough level
borough_gdf = map_gdf.dissolve(by='Borough').reset_index()

# Get top underpredicted boroughs by mean residual
top_under = (
    map_gdf.groupby('Borough')['residual_A']
    .mean()
    .sort_values(ascending=False)
    .head(5)
    .index.tolist()
)

top_over = (
    map_gdf.groupby('Borough')['residual_A']
    .mean()
    .sort_values(ascending=True)
    .head(5)
    .index.tolist()
)

# ── Plot ───────────────────────────────────────────────────────
fig, ax = plt.subplots(figsize=(10, 8), facecolor='#F7F4ED')
ax.set_facecolor('#F7F4ED')

gla_plot.plot(ax=ax, color='#D8D3C8', edgecolor='none')

map_gdf.plot(
    column='residual_A',
    cmap=div_cmap,
    norm=norm,
    legend=True,
    linewidth=0.1,
    edgecolor='#C9C1B1',
    legend_kwds={
        'shrink':  0.6,
        'aspect':  25,
        'pad':     0.02,
        'format':  '%.1f',
        'label':   'Residual'
    },
    ax=ax,
    missing_kwds={'color': '#C9C1B1'}
)

gla_plot.boundary.plot(ax=ax, color='#1B2632', linewidth=1.0)

# ── Highlight top underpredicted boroughs ──────────────────────
under_gdf = borough_gdf[borough_gdf['Borough'].isin(top_under)]
over_gdf  = borough_gdf[borough_gdf['Borough'].isin(top_over)]

under_gdf.boundary.plot(
    ax=ax, color='#A35139', linewidth=1.8, linestyle='--'
)
over_gdf.boundary.plot(
    ax=ax, color='#2C3B4D', linewidth=1.8, linestyle='--'
)
```

```python
# ── Borough centroid labels ──────────────────────────────────
for _, row in under_gdf.iterrows():
    cx, cy = row.geometry.centroid.x, row.geometry.centroid.y
    ax.annotate(
        row['Borough'],
        xy=(cx, cy),
        fontsize=7, color='#A35139',
        fontweight='bold', ha='center', va='center',
        bbox=dict(boxstyle='round,pad=0.2',
                  facecolor='#F7F4ED',
                  edgecolor='none', alpha=0.7)
    )

for _, row in over_gdf.iterrows():
    cx, cy = row.geometry.centroid.x, row.geometry.centroid.y
    ax.annotate(
        row['Borough'],
        xy=(cx, cy),
        fontsize=7, color='#2C3B4D',
        fontweight='bold', ha='center', va='center',
        bbox=dict(boxstyle='round,pad=0.2',
                  facecolor='#F7F4ED',
                  edgecolor='none', alpha=0.7)
    )

# ── Legend for highlights ────────────────────────────────────

legend_elements = [
    Line2D([0], [0], color='#A35139', linewidth=1.8,
           linestyle='--', label='Top 5 underpredicted boroughs'),
    Line2D([0], [0], color='#2C3B4D', linewidth=1.8,
           linestyle='--', label='Top 5 overpredicted boroughs'),
]
ax.legend(handles=legend_elements, fontsize=8,
          framealpha=0.8, labelcolor='#1B2632',
          loc='lower left', facecolor='#F7F4ED',
          edgecolor='#C9C1B1')

# ── Bounds ───────────────────────────────────────────────────
minx, miny, maxx, maxy = gla_plot.total_bounds
pad_x = (maxx - minx) * 0.02
pad_y = (maxy - miny) * 0.02
ax.set_xlim(minx - pad_x, maxx + pad_x)
ax.set_ylim(miny - pad_y, maxy + pad_y)

ax.set_title('Spatial residuals — Type A person/society crime',
             fontsize=11, fontweight='bold', color='#1B2632', pad=10)
ax.text(0.5, 1.005,
        'Rust = underpredicted · Blue = overpredicted · '
        'Dashed outlines = top 5 boroughs by mean residual',
        transform=ax.transAxes, ha='center',
        fontsize=8, color='#615D56', style='italic')
ax.set_axis_off()

plt.tight_layout()
plt.show()
```

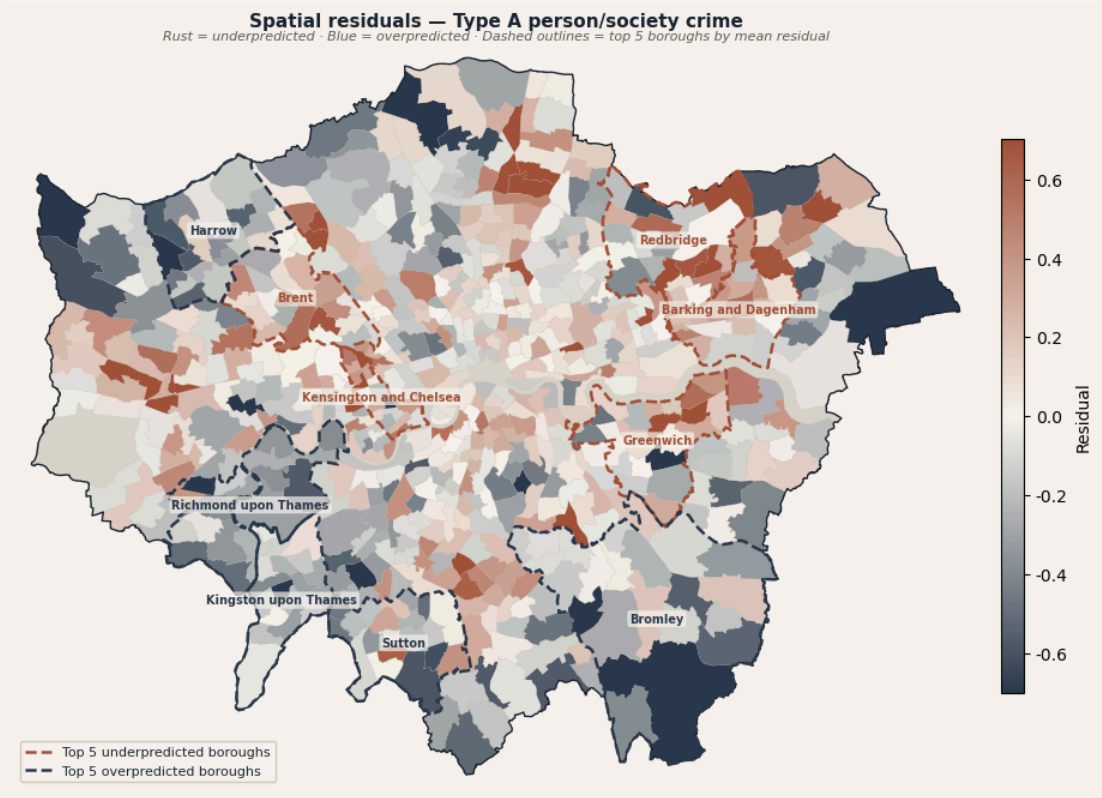


Figure 13: Spatial Residuals — Type A Person and Society Crime *Random Forest residuals across 679 London wards. Rust indicates underprediction; blue indicates overprediction. Dashed outlines mark the*

top five boroughs by mean residual in each direction. Underprediction concentrates in outer east and west London, where unmeasured structural conditions likely drive crime beyond what amenity configuration predicts. Kensington and Chelsea also underpredicts, consistent with high-value tourist targets generating crime independently of local amenity density. Overprediction clusters in outer south-west London, where amenity presence does not translate to elevated crime.

Limitations

[go back to the top]

First, police-recorded data systematically undercounts crime in deprived areas, likely compounding model underprediction there. Second, crime counts are aggregated to ward level while POI data is extracted at point level — this scale mismatch means within-ward variation in amenity distribution is lost. Third, OSM coverage is uneven across London, potentially underrepresenting amenity density in outer boroughs. Fourth, future work could explore gradient boosting approaches such as XGBoost, which may handle the zero-inflated distributions of sparse POI categories more effectively than Random Forest. Fifth, the absence of temporal, demographic, and socioeconomic controls limits causal inference. Sixth, ward boundaries are administratively defined and subject to the Modifiable Areal Unit Problem — findings may differ at alternative spatial scales. Seventh, 25 wards in the 2024 ONS boundary revision had no matching MPS crime records, reducing the analytical sample from 704 to 679 wards.

Conclusion

[go back to the top]

This study demonstrates that open-source POI data alone can predict ward-level crime density with moderate to strong accuracy — without demographic or socioeconomic variables. This matters because population-based indicators risk conflating the built environment with residents' characteristics, introducing assumptions about who lives where rather than what activities a place generates. By grounding predictions in amenity configuration rather than residential population, the model identifies areas vulnerable to specific crime types based on what a place does, not who lives there.

The key finding is that amenity type matters as much as amenity density. Convenience stores and fast food are the primary generators of person and society crime; cafés drive property crime. Most strikingly, night-time economy venues showed unexpectedly weak predictive power — suggesting that the regulatory environment of licensed premises suppresses crime opportunity despite high footfall.

The spatial residuals reveal where this framework reaches its limits. Systematic underprediction in outer boroughs indicates that socioeconomic conditions generate crime independently of physical land use — and that these are precisely the areas where police-recorded data is most likely to undercount. Future work integrating deprivation indices alongside POI density would strengthen both predictive accuracy and the identification of underserved areas where intervention is most needed.

In [88]:
```
elapsed = time.time() - start_time
print(f"Runtime: {elapsed/3600:.2f} hours ({elapsed/60:.1f} minutes)")
```

```
Runtime: 0.01 hours (0.6 minutes)
```

References

[go back to the top]

Boeing, G. (2017) 'OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks', *Computers, Environment and Urban Systems*, 65, pp. 126–139.

Brantingham, P. and Brantingham, P. (1995) 'Criminality of place: Crime generators and crime attractors', *European Journal on Criminal Policy and Research*, 3(3), pp. 5–26.

Cichosz, P. (2020) 'Urban crime risk prediction using point of interest data', *ISPRS International Journal of Geo-Information*, 9(7), 459. doi:10.3390/ijgi9070459.

Cohen, L.E. and Felson, M. (1979) 'Social change and crime rate trends: A routine activity approach', *American Sociological Review*, 44(4), pp. 588–608.

Malleson, N. and Andresen, M.A. (2016) 'Exploring the impact of ambient population measures on London crime hotspots', *Journal of Criminal Justice*, 46, pp. 52–63.

ONS Open Geography Portal (2024) *Ward boundary polygons*. Available at: https://geoportal.statistics.gov.uk (Accessed: April 2025).